

SLIM SERIES · 02

느린 책방을 짓다

라라벨을 지나온 PHP 개발자를 위한 심포니 8.0 안내서
Symfony 8.0 · PHP 8.5 · 100면

차례 (1)

머리말	4
1부 — 두 풍경	
1. 라라벨이 길러준 직관	6
2. 심포니라는 풍경	7
3. PHP 8.5라는 시대	8
4. 의존성 주입의 두 풍경	9
5. ORM의 두 풍경	10
6. 품의 두 풍경	11
7. 환경설정의 두 풍경	12
2부 — 빈 가게	
8. 시작	14
9. 디렉터리 읽기	15
10. 도커로 데이터베이스 띄우기	16
11. .env와 환경 분리	17
12. 첫 라우트	18
13. 첫 컨트롤러	19
14. 요청의 일생	20
15. AbstractController의 도구들	21
16. base.html.twig	22
3부 — 진열	
17. 도메인 — Product와 Category	24
18. ER 다이어그램	25
19. Product 엔티티	26
20. Category와 ManyToMany	27
21. ProductImage	28
22. 마이그레이션	29
23. 픽스처로 시드 만들기	30
24. ProductRepository — QueryBuilder	31
25. 카탈로그 컨트롤러	32
26. 카탈로그 템플릿	33
27. 페이지네이션	34
28. 상품 상세 페이지	35
29. 검색	36
30. 트윅 매크로	37

차례 (2)

4부 — 장바구니

31. 두 가지 장바구니	39
32. Cart 도메인	40
33. 장바구니의 상태	41
34. CartFactory — 손님과 회원	42
35. 담기	43
36. 빼기와 수량 변경	44
37. 합계 계산 — Money 객체	45
38. 장바구니 머지	46
39. 미니 카트 위젯	47
40. 빈 장바구니의 친절	48
41. 재고를 흘리지 않게	49

5부 — 사람과 자물쇠

42. Symfony Security 풍경	51
43. User 엔티티	52
44. 비밀번호 해싱	53
45. 회원가입 폼	54
46. 로그인 폼	55
47. firewall.yaml 핵심	56
48. ROLE 관리	57
49. #[IsGranted]	58
50. Voter — 도메인 권한	59
51. 비밀번호 재설정	60
52. 로그아웃과 세션	61

6부 — 주문

53. Order 도메인	63
54. ER — 주문 데이터	64
55. 주문의 상태	65
56. 다단계 폼	66
57. 배송 주소 폼	67
58. 결제 폼	68
59. Cart에서 Order로	69
60. 트랜잭션	70
61. Workflow 컴포넌트	71
62. 주문 메일 (Mailer)	72
63. Messenger — 비동기	73
64. Scheduler — 자동화	74
65. 주문 내역 보기	75

7~10부

관리자, 화면, 운영, 닫는 글	76~
-------------------------	-----

머리말

이 책은 라라벨을 충분히 만져본 PHP 개발자를 위해 쓰였다. 우리는 이미 컨트롤러를 짓고, 마이그레이션을 만들고, 큐로 메일을 보낸다. 그 손이 심포니라는 다른 도시에 닿았을 때 우왕좌왕하지 않도록, 길의 폭과 길의 이름을 적어 두는 일이 이 책의 일이다.

라라벨은 친절할 동네다. 길이 잘 닦여 있고, 어디에 무엇이 있는지 보이지 않는 손이 미리 정리해 둔다. 그 친절함이 우리를 빠르게 자라게 했다.

심포니는 그보다 조금 더 조용한 동네다. 길이 적게 닦여 있다는 뜻이 아니라, 우리가 길을 직접 고를 수 있다는 뜻이다. 라라벨이 거실이라면, 심포니는 가구 없이 비워 둔 큰 방에 가깝다. 어디든 책상을 둘 수 있다.

백 쪽은 한 도시의 모든 길을 안내하기에는 짧다. 그래서 이 책은 한 가지 일을 함께 한다. 작은 책방을 짓는 것이다. 이름은 '느린 책방'.

책방은 작지만 보통의 가게가 가지고 있는 모든 것을 가지고 있다. 책장이 있고, 가격표가 있고, 장바구니가 있고, 계산대가 있고, 손님이 있고, 주인이 있다. 그래서 우리가 만들 가게에는 컨트롤러가 있고, 엔티티가 있고, 폼이 있고, 보안이 있고, 큐가 있다.

일을 마치고 나면 두 가지가 남는다. 하나는 작동하는 코드 한 폴더, 다른 하나는 심포니의 골격에 대한 감각이다. 두 번째가 사실은 더 오래 남는다.

코드는 읽을 만큼만 적는다. 명령어는 따라칠 만큼만 적는다. 다이어그램은 머릿속에 남을 만한 것만 그린다. 나머지 시간은 생각하는 데 쓰자.

1부

두 풍경

1. 라라벨이 길러준 직관

라라벨에서 우리는 이미 많은 것을 배웠다. `Route::get`으로 길을 깔고, 컨트롤러에서 Eloquent로 데이터를 꺼내고, Blade로 화면을 그렸다. `php artisan make:` 한 줄이면 새 파일이 자리 잡았고, `.env` 한 줄을 바꾸면 모든 환경이 따라 움직였다. 큐로 메일을 보내고, Schedule로 매일 새벽에 일을 깨웠다.

이 직관 대부분은 심포니에서도 그대로 통한다. 다만 이름이 다르고, 위치가 다르다.

라라벨에서 `Route::get`은 심포니에서 `#[Route]`라는 PHP 속성이다. Eloquent는 Doctrine으로, Blade는 Twig으로, `artisan`은 `bin/console`로 옮긴다. `Mail::to()`는 `Mailer::send()`다. 큐는 Messenger, 스케줄러는 Scheduler. 이름표만 바꿔 단 셈이다.

다른 것은 두 가지다.

첫째, 심포니는 마법을 덜 쓴다. `User::find(1)` 대신 `$users->find(1)`이라고 쓴다. 받아쓴 객체의 출처가 코드에 적혀 있다. 처음엔 답답하고, 나중에는 안전하다.

둘째, 심포니는 결정을 미룬다. 라라벨이 좋은 기본값을 정해 두는 것이라면, 심포니는 우리에게 결정을 묻는다. 그 결정이 점점 쌓여서 나만의 도시가 된다.

이 책은 그 도시의 작은 거리 하나를 함께 걸어 보는 일이다.

2. 심포니라는 풍경

심포니의 한가운데에는 다섯 개의 사물이 있다.

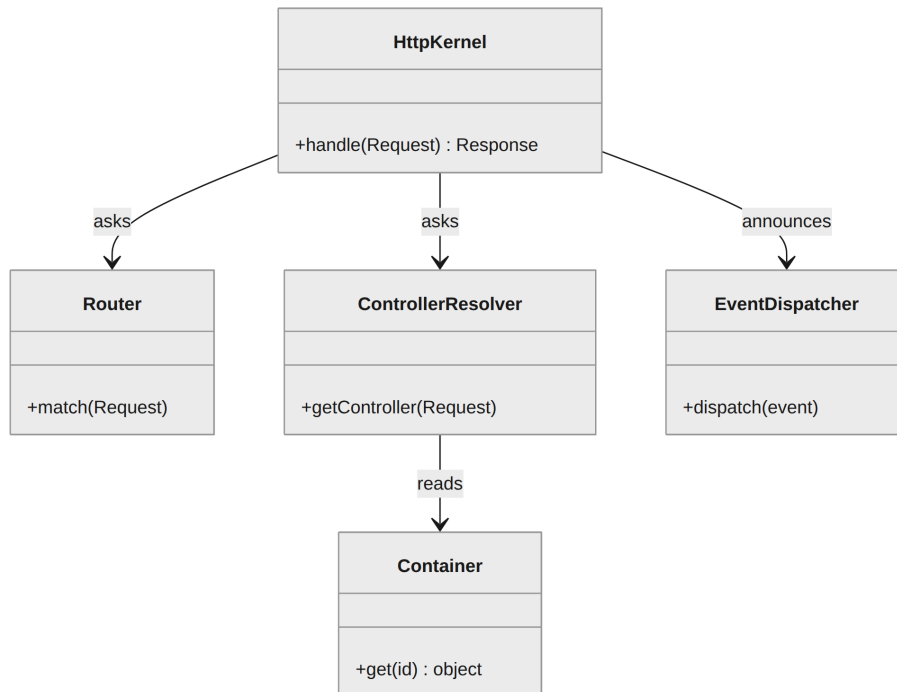


그림 1. 한 요청을 둘러싼 다섯 사물

HttpKernel은 모든 요청을 받는 손이다. 우리는 컨트롤러를 찾고 라우트를 정하지만, 그 모든 것이 모이는 곳은 결국 이 클래스의 `handle()` 메서드다.

Router는 URL을 컨트롤러로 옮기는 사전이다. **ControllerResolver**는 그 컨트롤러 객체를 실제로 꺼내는 일을 한다. 객체를 꺼내려면 의존성도 함께 꺼내야 하는데, 그 일은 **Container**가 한다. 처음부터 끝까지, 어떤 단계에 도달할 때마다 **EventDispatcher**가 모든 구독자에게 그 사실을 알린다.

이름이 많지만, 이름들이 하는 일은 단순하다. 받고, 옮기고, 꺼내고, 실행하고, 알린다. 라라벨에도 같은 일이 있었다. 이름만 달랐을 뿐이다.

3. PHP 8.5라는 시대

이 책은 PHP 8.5.5를 기준으로 쓴다. 2026년 3월에 나온 안정 버전이다. 8.5는 작지만 즐거운 변화를 가져왔다.

파이프 연산자

```
$slug = $title
    |> strtolower(...)
    |> trim(...)
    |> fn ($s) => str_replace(' ', '-', $s);
```

위에서 아래로 읽힌다. 라라벨의 `Str::of()->lower()->trim()->slug()` 체인이 코어 문법으로 들어왔다고 보면 된다. 우리 책방에서는 책 제목으로 슬러그를 만들 때 자주 쓸 것이다.

clone with

```
$shipped = clone $order with {
    status: 'shipped',
    shippedAt: new DateTimeImmutable(),
};
```

원본은 그대로 두고 새 객체만 바꾼다. `readonly` 클래스와 함께 쓸 때 가장 빛난다. 함수형의 깔끔함을 객체지향 안에 들여놓은 셈이다.

그 외

`#[NoDiscard]` 속성, URI 확장, Lazy Object 안정화. 우리 책방의 코드 한 곳에서는 만나게 된다.

4. 의존성 주입의 두 풍경

라라벨에서 우리는 자주 이렇게 적었다.

```
class OrderController extends Controller
{
    public function __construct(
        protected OrderService $orders,
    ) {}
}
```

심포니에서도 같다. 다만 한 가지 다르다. 심포니의 컨테이너는 우리가 적은 거의 모든 클래스를 자동으로 알아본다.

`config/services.yaml`의 단 한 줄 덕분이다.

```
App\:  
    resource: '../src/'  
    autowire: true  
    autoconfigure: true
```

`autowire`는 인자의 타입을 보고 의존성을 채운다. `autoconfigure`는 클래스가 이벤트 구독자나 콘솔 명령처럼 보이면 자동으로 그 역할을 등록한다. 라라벨의 서비스 컨테이너도 비슷하지만, 심포니는 그 자동 처리를 한 발 더 나아가게 한다.

그래서 심포니의 코드는 보통 `__construct`에 인자를 적기만 하면 된다. 컨테이너가 알아서 와서 자리에 앉는다. 메모해 둘 단어 하나는 `autowire`다.

5. ORM의 두 풍경

Eloquent는 모델 자체가 모든 일을 한다. `User::find(1)`로 꺼내고, `$user->save()`로 저장한다. 활성 레코드 (active record) 패턴이다.

Doctrine은 그 일을 두 클래스로 나누어 둔다.

```
// 데이터의 모양
#[ORM\Entity(repositoryClass: ProductRepository::class)]
class Product { /* 필드만 */ }

// 데이터의 행위
class ProductRepository extends ServiceEntityRepository
{
    public function findFeatured(): array { /* ... */ }
}
```

모델은 데이터, 리포지토리는 행위. 데이터 매퍼(data mapper) 패턴이라고 부른다. 처음에는 번거롭지만, 도메인이 자라날수록 고마워진다.

저장도 다르다. Eloquent의 `save()` 대신 심포니에서는 두 줄을 쓴다.

```
$em->persist($product);
$em->flush();
```

`persist`는 "이걸 기억해 뒀", `flush`는 "지금 데이터베이스에 써". 트랜잭션 안에서 묶을 수 있다는 뜻이다.

6. 폼의 두 풍경

라라벨에서 폼은 보통 `FormRequest` 로 검증하고 컨트롤러에서 직접 객체를 만든다.

```
class StoreProduct extends FormRequest
{
    public function rules(): array
    {
        return ['name' => 'required|max:200'];
    }
}
```

심포니는 그 자리에 `FormType` 이라는 클래스를 둔다.

```
class ProductType extends AbstractType
{
    public function buildForm($b, $opts): void
    {
        $b->add('name', TextType::class);
    }

    public function configureOptions($r): void
    {
        $r->setDefaults(['data_class' => Product::class]);
    }
}
```

차이는 두 가지다. 첫째, 검증 규칙은 폼이 아니라 엔티티의 `#[Assert]` 속성에 둔다. 둘째, 폼이 직접 객체를 만든다. 컨트롤러는 그저 `handleRequest()` 한 줄만 부르면 된다. 폼이 두 일을 함께 하니까 컨트롤러가 짧아진다.

이 차이가 처음엔 어색하지만, 같은 폼을 두 곳에서 쓰는 순간 고마워진다.

7. 환경설정의 두 풍경

라라벨의 `.env`와 `config/*.php`는 우리에게 익숙하다. 심포니도 같은 자리에 같은 약속을 둔다. `.env`로 비밀과 환경을, `config/*.php`로 구조를.

심포니 8부터는 YAML이 기본 자리에서 물러났다. `config/packages/twig.php`는 이렇게 생겼다.

```
use Symfony\Config\TwigConfig;

return static function (TwigConfig $twig) {
    $twig->defaultPath('%kernel.project_dir%/templates');
    $twig->strictVariables(true);
};
```

타입이 있는 PHP다. IDE가 자동완성을 한다. 오타가 컴파일 시점에 잡힌다. 라라벨의 `config/app.php`가 단순한 배열이었다면, 심포니는 같은 일을 객체와 메서드로 한다.

환경별 분리는 거의 같다. `.env`는 모두에게 공통, `.env.dev`·`.env.prod`가 환경별. `.env.local`은 자기 컴퓨터만의 비밀. 라라벨에서 익혔던 그 약속 그대로다.

2부

빈 가게

8. 시작

심포니는 자신의 CLI를 가지고 있다. 한 번 깔아 두면 두 번 다시 신경 쓸 일이 없다.

```
curl -sS https://get.symfony.com/cli/installer | bash
```

설치가 끝났다면 환경을 점검한다. PHP 8.4 이상이 필요하다. 우리는 8.5.5를 쓴다.

```
symfony check:requirements
```

문제가 없다면 새 프로젝트를 만든다. `--webapp`은 웹 애플리케이션에 필요한 패키지를 함께 받는다는 뜻이다.

```
symfony new slow-bookstore \
  --version="8.0.*" --webapp
cd slow-bookstore
symfony serve -d
```

`-d`는 데몬으로 띄운다는 뜻. 브라우저로 `https://127.0.0.1:8000`을 열면 환영 페이지가 보인다. 라라벨의 `php artisan serve`와 같은 첫 인상이다.

이제 우리에게 빈 가게 한 채가 생겼다. 다음 일은 그 가게에 책장을 들이는 일이다.

9. 디렉터리 읽기

새 프로젝트의 폴더는 이렇게 생겼다.

```
slow-bookstore/
├─ bin/console      # artisan에 해당
├─ config/          # 라우트, 서비스 등의 설정
├─ public/          # 웹 루트, index.php
├─ src/             # 우리가 쓸 PHP 코드
├─ templates/       # Twig 템플릿
├─ migrations/      # Doctrine 마이그레이션
├─ translations/    # 번역 파일
├─ var/             # 캐시, 로그
├─ vendor/          # composer 의존성
├─ compose.yaml     # 도커 컴포즈
└─ .env             # 환경 변수
```

라라벨과 거의 같다. 다른 점은 두 가지. `app/` 대신 `src/` 이고, `routes/` 폴더가 없다. 라우트는 컨트롤러의 PHP 속성으로 직접 박는다.

`config/` 안의 파일들을 한 번 열어 보길 권한다. 심포니 8부터는 YAML이 기본 자리에서 물러났고, PHP 배열로 작성된 설정이 보일 것이다. 우리가 쓰는 PHP가 설정도 같이 한다.

10. 도커로 데이터베이스 띄우기

`--webapp` 옵션을 주면 `compose.yaml`이 함께 만들어진다. 우리는 PostgreSQL을 쓸 것이다.

```
# compose.yaml
services:
  database:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: app
      POSTGRES_PASSWORD: !ChangeMe!
      POSTGRES_USER: app
    ports: ["5432:5432"]
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

한 줄이면 띄운다.

```
docker compose up -d
```

심포니 CLI는 도커가 도는 것을 보면 `.env`의 `DATABASE_URL`을 자동으로 만들어 준다. `symfony var:export --multiline`으로 확인할 수 있다.

라라벨의 Sail과 비슷하지만, 한 발 가볍다. 도커는 데이터베이스만 띄우고, PHP는 우리 컴퓨터에서 직접 돈다. 그래서 빠르다.

11. .env와 환경 분리

심포니의 `.env` 분할은 라라벨보다 한 발 더 정교하다. 네 개의 파일이 있다.

- `.env` — 모두에게 공통. 커밋한다.
- `.env.dev` / `.env.prod` / `.env.test` — 환경별. 커밋한다.
- `.env.local` — 자기 컴퓨터만의 비밀. `.gitignore`로 막는다.
- `.env.{env}.local` — 환경별 + 자기만의 비밀.

읽는 순서가 정해져 있다. 더 구체적인 파일이 위에 덮인다. 그래서 우리는 비밀 키 같은 것을 `.env.local`에만 두고 나머지는 가짜 기본값을 둔다.

코드에서는 한 줄이면 꺼낸다.

```
$dsn = $_ENV['DATABASE_URL'];
```

또는 자동 주입으로.

```
public function __construct(  
    #[Autowire('%env(DATABASE_URL)%')]  
    private string $dsn,  
    ) {}
```

같은 일을 두 가지 방법으로 한다. 두 번째가 테스트하기에 더 친절하다.

12. 첫 라우트

가장 작은 컨트롤러를 만들어 보자. 우리 책방의 환영 페이지다.

```
// src/Controller/HomeController.php
namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

class HomeController extends AbstractController
{
    #[Route('/', name: 'home')]
    public function __invoke(): Response
    {
        return new Response('느린 책방에 오신 걸 환영합니다.');
```

브라우저에서 `/`를 열면 그 한 줄이 보인다. 라라벨로 치면 다음과 같다.

```
Route::get('/', fn () => '환영합니다.')->name('home');
```

다른 점이 한눈에 들어온다. 라라벨은 길과 응답을 한 곳에 묶고, 심포니는 그 둘을 클래스의 PHP 속성으로 묶는다. 어느 쪽이 옳다고 말하기 어렵다. 다만 심포니의 방식은 컨트롤러가 자기 길을 직접 안다는 뜻이다.

13. 첫 컨트롤러

심포니 8에서 가장 권장하는 컨트롤러 모양은 메서드 하나짜리 클래스다. `__invoke`라는 마법 메서드를 쓴다.

```
class ShowProductController extends AbstractController
{
    #[Route('/books/{slug}', name: 'product_show')]
    public function __invoke(
        string $slug,
        ProductRepository $products,
    ): Response {
        $product = $products->findBySlug($slug);
        if (!$product) {
            throw $this->createNotFoundException();
        }
        return $this->render('product/show.html.twig', [
            'product' => $product,
        ]);
    }
}
```

이 컨트롤러는 곧 하나의 함수다. 책 한 권을 보여주는 일, 그 한 가지만 한다. 책을 만드는 일, 책을 지우는 일은 다른 클래스다. 한 클래스가 한 가지 일을 한다는 약속은, 그 클래스를 다시 읽었을 때의 친절함으로 돌아온다.

라라벨 8부터의 `invokable controller`와 같은 발상이다. 심포니에서는 그것이 기본이다.

14. 요청의 일생

한 요청이 들어와 응답이 되어 나가는 길은 짧지 않다.

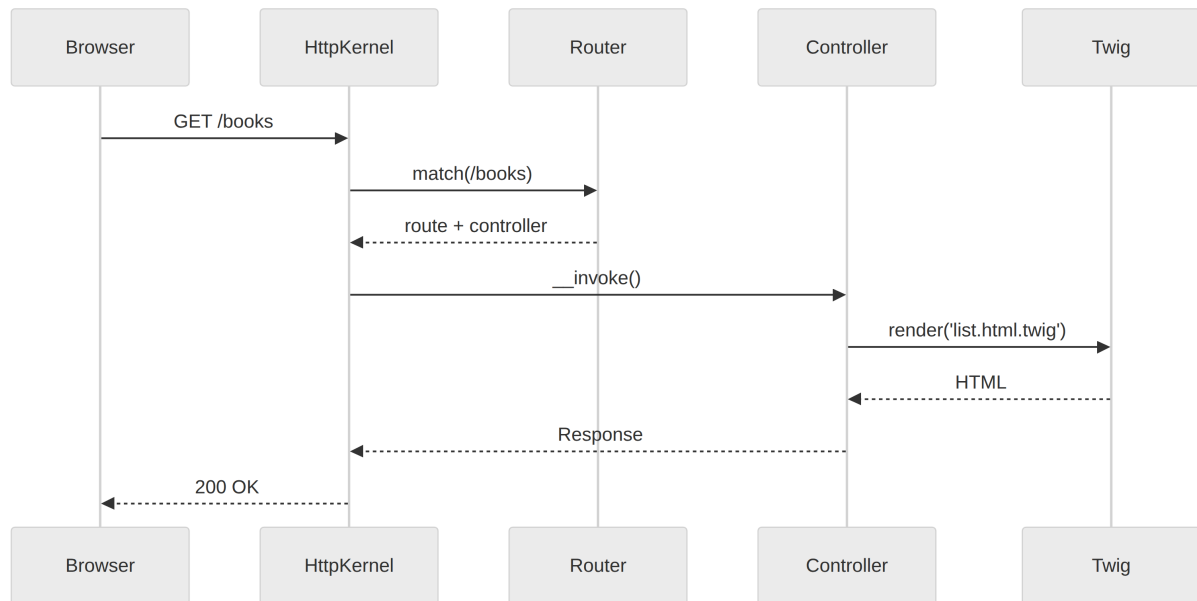


그림 2. 요청 하나가 거치는 다섯 손

`HttpKernel`이 받고, `Router`에게 길을 묻고, 그 길을 따라 컨트롤러를 깨운다. 컨트롤러는 자기 일을 마치고 `Twig`에게 화면을 부탁한 뒤, `Response`를 손에 쥐고 다시 `HttpKernel`에게 돌아온다. 그제서야 응답이 브라우저로 떠난다.

읽고 나면 단순하다. 다만 이 길의 어느 지점에서든 `EventDispatcher`가 사람들을 깨운다는 점만 기억하자. 우리는 그 사람들 중 하나가 되어, 우리만의 일을 슬쩍 끼워 넣을 수 있다.

15. AbstractController의 도구들

`AbstractController`를 상속하면 자주 쓰는 도구가 손에 들어온다. 외워 둘 만한 다섯 가지를 적는다.

```
// HTML 응답
return $this->render('p/show.html.twig', ['p' => $p]);

// JSON 응답
return $this->json(['ok' => true]);

// 다른 라우트로
return $this->redirectToRoute('home');

// 404
throw $this->createNotFoundException();

// 한 번만 보여줄 메시지
$this->addFlash('success', '답았습니다.');
```

라라벨의 `view()`, `response()->json()`, `redirect()`, `abort(404)`, `session()->flash()`와 거의 일대일이다. 같은 손이 같은 일을 한다.

이 다섯 가지면 컨트롤러의 90%가 끝난다. 나머지 10%가 이 책의 다른 페이지에서 등장한다.

16. base.html.twig

한 사이트의 모든 페이지가 공유할 뼈대를 적어 두자. `templates/base.html.twig` 이다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}느린 책방{% endblock %}</title>
  {% block stylesheets %}{% importmap('app') %}{% endblock %}
</head>
<body>
  <header>
    <a href="{{ path('home') }}">느린 책방</a>
    <a href="{{ path('cart_show') }}">
      장바구니 ({{ cart_count() }})</a>
  </header>
  {% for label, msgs in app.flashes %}
    {% for m in msgs %}
      <div class="flash flash-{{ label }}">{{ m }}</div>
    {% endfor %}
  {% endfor %}
  <main>{% block body %}{% endblock %}</main>
</body>
</html>
```

자식 템플릿은 `extends` 로 이 뼈대를 받는다. 라라벨의 `@extends` 와 같다. 다만 Twig은 PHP를 직접 쓰지 못한다. 메서드 호출은 `app.flashes` 처럼 점 하나로 충분하다.

3부

진 열

17. 도메인 — Product와 Category

책방의 가장 작은 사물은 책 한 권이다. 그것을 우리는 Product라고 부른다.

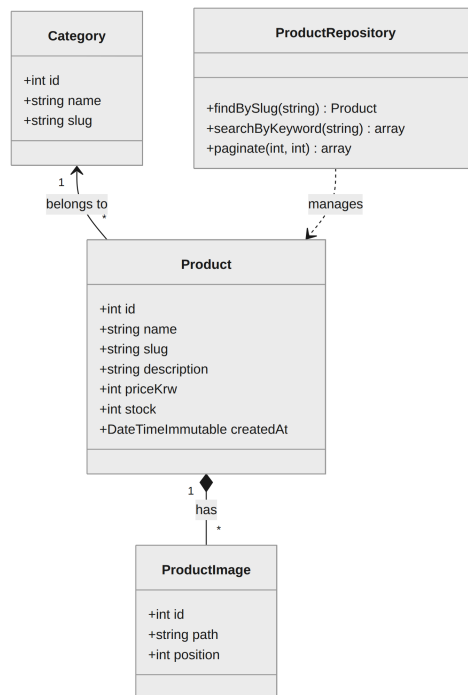


그림 3. 진열장의 클래스 지도

Product는 책 자체. **Category**는 분류 (소설, 시, 에세이...). **ProductImage**는 표지와 본문 사진을 담는 작은 사물이다.

관계를 보자. **Category** 하나는 여러 **Product**를 가질 수 있고, **Product** 하나는 여러 **ProductImage**를 가질 수 있다. 그림에서 1과 * 표기로 보이는 것이다.

이 세 클래스가 진열장을 지탱한다. 그 외의 모든 일—검색, 페이지네이션, 정렬—은 **ProductRepository**가 한다. 데이터는 명사, 행위는 동사. 명사를 한 클래스에 두고, 동사를 다른 클래스에 두는 일이 Doctrine의 약속이다.

18. ER 다이어그램

같은 그림을 데이터베이스의 시점에서 보면 다음과 같다.

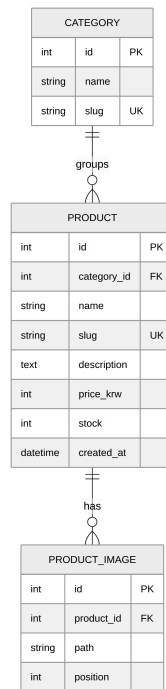


그림 4. 진열장 데이터의 모양

세 테이블, 두 외래 키. 외래 키 하나가 부모 자식을 묶는다. `category_id`는 책이 어느 분류에 속하는지를 가리키고, `product_id`는 이미지가 어느 책의 것인지를 가리킨다.

이 그림은 우리가 두 번째 학기에 이미 그려본 것이다. 데이터베이스 강의의 첫 그림. 심포니에서도 같다. 사물은 변하지 않았고, 사용하는 도구만 바뀌었다.

실제 마이그레이션은 다음 페이지부터 우리가 직접 만들 것이다. 손으로 SQL을 적지는 않는다. Doctrine이 우리가 적은 엔티티를 보고 알아서 그려준다.

19. Product 엔티티

엔티티는 데이터의 모양만 담는다. 행위는 리포지토리가 한다.

```
// src/Entity/Product.php
namespace App\Entity;

use App\Repository\ProductRepository;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Validator\Constraints as Assert;

#[ORM\Entity(repositoryClass: ProductRepository::class)]
class Product
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 200)]
    #[Assert\NotBlank, Assert\Length(max: 200)]
    private string $name = '';

    #[ORM\Column(length: 220, unique: true)]
    private string $slug = '';

    #[ORM\Column(type: 'text')]
    private string $description = '';

    #[ORM\Column]
    #[Assert\PositiveOrZero]
    private int $priceKrw = 0;

    #[ORM\Column]
    private int $stock = 0;

    #[ORM\Column]
    private DateTimeImmutable $createdAt;

    public function __construct() {
        $this->createdAt = new DateTimeImmutable();
    }
}
```

게터·세터는 IDE가 만들어 준다. 검증 규칙이 데이터의 자리에 있는 점에 주목하자.

20. Category와 ManyToMany

책 한 권은 보통 여러 분류에 동시에 속한다. '시'이면서 '한국 문학'일 수 있다. 그래서 ManyToMany를 쓴다.

```
#[ORM\Entity]
class Category
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 80)]
    private string $name = '';

    #[ORM\Column(length: 80, unique: true)]
    private string $slug = '';
}

// Product 안에 추가
#[ORM\ManyToMany(targetEntity: Category::class)]
#[ORM\JoinTable(name: 'product_category')]
private Collection $categories;

public function __construct() {
    $this->categories = new ArrayCollection();
    $this->createdAt = new DateTimeImmutable();
}
```

Doctrine은 자동으로 `product_category` 조인 테이블을 만든다. 라라벨의 `belongsToMany`와 같은 일을 한다. 조인 테이블 이름을 직접 짓고 싶다면 `JoinTable` 속성을 쓴다.

다음 페이지에서 우리는 책 한 권에 표지를 붙인다.

21. ProductImage

책의 표지·내지 사진은 별개의 사물로 둔다. 한 책이 여러 사진을 가질 수 있고, 정렬 순서가 필요하기 때문이다.

```
#[ORM\Entity]
class ProductImage
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    private ?int $id = null;

    #[ORM\ManyToOne(inversedBy: 'images')]
    #[ORM\JoinColumn(nullable: false)]
    private Product $product;

    #[ORM\Column(length: 255)]
    private string $path = '';

    #[ORM\Column]
    private int $position = 0;
}

// Product 안에 추가
#[ORM\OneToMany(
    targetEntity: ProductImage::class,
    mappedBy: 'product',
    orphanRemoval: true,
    cascade: ['persist'],
)]
#[ORM\OrderBy(['position' => 'ASC'])]
private Collection $images;
```

`orphanRemoval`은 부모에서 떼어 놓인 이미지를 자동으로 지운다. `cascade: persist`는 부모를 저장할 때 자식도 함께 저장한다. 이 두 옵션이 우리의 손을 가볍게 해 준다.

22. 마이그레이션

엔티티 세 개를 적었으니 데이터베이스에 옮길 차례다.

```
php bin/console make:migration
```

심포니가 우리가 적은 엔티티와 현재 스키마를 비교하여, 차이만큼만 자동으로 마이그레이션 파일을 그린다. migrations/ 폴더에 Version20260503...php 같은 이름으로.

그 파일을 한 번 열어 보길 권한다. up() 과 down() 안에 SQL이 적혀 있다. 우리가 직접 적지 않았지만, 우리가 적은 PHP에서 그대로 나왔다는 사실을 눈으로 확인하면 안심이 된다.

적용은 한 줄.

```
php bin/console doctrine:migrations:migrate
```

한 번 더 묻는다. yes를 치면 끝이다. 라라벨의 php artisan migrate와 같은 손이다.

나중에 엔티티를 고칠 때마다 같은 두 줄을 부르면 된다. 마이그레이션 파일은 코드와 함께 git에 커밋하자. 동료의 데이터베이스가 우리의 것과 같은 모양이 되도록.

23. 픽스처로 시드 만들기

빈 가게는 외롭다. 시연용 데이터를 한 줄 깔자. 라라벨의 Seeder에 해당하는 도구가 심포니에는 DoctrineFixturesBundle이라는 이름으로 있다.

```
composer require --dev doctrine/doctrine-fixtures-bundle
```

```
// src/DataFixtures/AppFixtures.php
namespace App\DataFixtures;

use App\Entity\Category;
use App\Entity\Product;
use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;

class AppFixtures extends Fixture
{
    public function load(ObjectManager $em): void
    {
        $poetry = (new Category())
            ->setName('시')->setSlug('poetry');
        $em->persist($poetry);

        for ($i = 1; $i <= 12; $i++) {
            $p = (new Product())
                ->setName("느린 시 $i")
                ->setSlug("slow-poem-$i")
                ->setPriceKrw(12000)
                ->setStock(20);
            $p->getCategories()->add($poetry);
            $em->persist($p);
        }
        $em->flush();
    }
}
```

```
php bin/console doctrine:fixtures:load
```

24. ProductRepository — QueryBuilder

리포지토리는 데이터에 묻는 곳이다. 자주 쓰는 질의는 메서드로 적어 둔다.

```
// src/Repository/ProductRepository.php
namespace App\Repository;

use App\Entity\Product;
use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepository;
use Doctrine\Persistence\ManagerRegistry;

class ProductRepository extends ServiceEntityRepository
{
    public function __construct(ManagerRegistry $r)
    {
        parent::__construct($r, Product::class);
    }

    public function findBySlug(string $slug): ?Product
    {
        return $this->findOneBy(['slug' => $slug]);
    }

    public function searchByKeyword(string $q): array
    {
        return $this->createQueryBuilder('p')
            ->andWhere('p.name LIKE :q')
            ->setParameter('q', '%'.$q.'%')
            ->orderBy('p.createdAt', 'DESC')
            ->setMaxResults(50)
            ->getQuery()->getResult();
    }
}
```

Eloquent의 chain과 비슷하지만, 변수가 자리표(`:q`)로 분리되어 있는 점에 주목하자. SQL 인젝션은 그 자리에서 막힌다.

25. 카탈로그 컨트롤러

이제 진열대를 만든다. 첫 컨트롤러는 모든 책을 보여주는 일이다.

```
// src/Controller/CatalogController.php
namespace App\Controller;

use App\Repository\ProductRepository;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

class CatalogController extends AbstractController
{
    #[Route('/books', name: 'catalog')]
    public function __invoke(
        Request $request,
        ProductRepository $products,
    ): Response {
        $page = max(1, (int) $request->query->get('page', 1));
        $perPage = 12;

        $items = $products->paginate($page, $perPage);
        $total = $products->count([]);

        return $this->render('catalog/index.html.twig', [
            'products' => $items,
            'page' => $page,
            'total' => $total,
            'perPage' => $perPage,
        ]);
    }
}
```

`Request`를 인자에 적으면 자동으로 들어온다. `?page=3` 같은 쿼리 스트링은 `$request->query->get('page')`로 꺼낸다.

26. 카탈로그 템플릿

```
{# templates/catalog/index.html.twig #}
{% extends 'base.html.twig' %}
{% block title %}책 목록 - 느린 책방{% endblock %}
{% block body %}
    <h1>진열대</h1>
    <ul class="product-grid">
        {% for p in products %}
            <li>
                <a href="{{ path('product_show', {slug: p.slug}) }}">
                    {% if p.images|length > 0 %}
                        
                    {% endif %}
                    <h2>{{ p.name }}</h2>
                    <p>{{ p.priceKrw|number_format }}원</p>
                </a>
            </li>
            {% else %}
                <li>진열된 책이 없습니다.</li>
            {% endfor %}
        </ul>
        {{ include('_pagination.html.twig', {
            page: page, total: total, perPage: perPage,
        }) }}
    {% endblock %}
```

Twig의 `{% else %}`는 비어 있을 때를 우아하게 처리한다. 우리는 그 친절을 늘 잊지 않을 것이다.

27. 페이지네이션

외부 패키지 없이 손으로 짜 보자. 라라벨의 `paginate()`가 하던 일을 우리가 직접 한다.

```
// ProductRepository에 추가
public function paginate(int $page, int $perPage): array
{
    $offset = ($page - 1) * $perPage;
    return $this->createQueryBuilder('p')
        ->orderBy('p.createdAt', 'DESC')
        ->setFirstResult($offset)
        ->setMaxResults($perPage)
        ->getQuery()->getResult();
}
```

```
{# templates/_pagination.html.twig #}
{% set last = (total / perPage)|round(0, 'ceil') %}
<nav class="pagination">
    {% for n in 1..last %}
        {% if n == page %}
            <span class="current">{{ n }}</span>
        {% else %}
            <a href="?page={{ n }}">{{ n }}</a>
        {% endif %}
    {% endfor %}
</nav>
```

외부 패키지를 쓰지 않은 이유는, 페이지네이션은 보통 우리가 원하는 모양이 따로 있기 때문이다. 손이 따끔하지만, 결과가 자기 모양에 가깝다.

28. 상품 상세 페이지

책 한 권의 자기 페이지를 만든다. URL은 슬러그로 잡는다.

```
class ShowProductController extends AbstractController
{
    #[Route('/books/{slug}', name: 'product_show')]
    public function __invoke(
        string $slug,
        ProductRepository $products,
    ): Response {
        $product = $products->findBySlug($slug);
        if (!$product) {
            throw $this->createNotFoundException();
        }

        $related = $products->findRelatedTo($product, 4);

        return $this->render('product/show.html.twig', [
            'product' => $product,
            'related' => $related,
        ]);
    }
}
```

슬러그를 우리가 직접 받아서 리포지토리에 묻는다. 심포니에는 더 짧은 길도 있다. `#[MapEntity(mapping: ['slug' => 'slug'])]`를 쓰면 컨트롤러 인자에 바로 `Product $product`를 받을 수 있다. 라라벨의 라우트 모델 바인딩과 같다.

다만 첫 코드는 직접 묻는 길이 분명해서 좋다. 마법은 익숙해진 다음에 부르자.

29. 검색

가게에 책이 늘면 검색이 필요해진다. 가장 단순한 모양부터.

```
class SearchController extends AbstractController
{
    #[Route('/search', name: 'search')]
    public function __invoke(
        Request $request,
        ProductRepository $products,
    ): Response {
        $q = trim($request->query->get('q', ''));
        $hits = $q !== ''
            ? $products->searchByKeyword($q)
            : [];

        return $this->render('search.html.twig', [
            'q' => $q,
            'hits' => $hits,
        ]);
    }
}
```

`LIKE`는 책장이 작을 때만 쓸 수 있는 도구다. 책방이 커지면 PostgreSQL의 `tsvector`나 Meilisearch 같은 외부 검색 엔진으로 옮긴다. 심포니에는 `symfony/messenger`로 그것에 데이터를 흘려보내는 `SearchIndexer`를 만들 수 있다.

우리는 지금 작은 책방이다. `LIKE`로 충분하다. 도구는 필요할 때 부르는 편이 좋다.

30. 트윅 매크로

같은 마크업이 여러 페이지에서 반복되면 매크로로 만들자. 라라벨의 컴포넌트와 비슷한 자리다.

```
{# templates/_macros.html.twig #}
{% macro price(krw) %}
    <span class="price">
        {{ krw|number_format }}
        <small>원</small>
    </span>
{% endmacro %}

{% macro card(p) %}
    <article class="card">
        <h3>
            <a href="{{ path('product_show', {slug: p.slug}) }}">
                {{ p.name }}
            </a>
        </h3>
        {{ _self.price(p.priceKrw) }}
    </article>
{% endmacro %}
```

다른 템플릿에서는 import해서 부른다.

```
{% import '_macros.html.twig' as ui %}
{{ ui.price(p.priceKrw) }}
{{ ui.card(p) }}
```

매크로는 어디서든 같은 모양으로 가격을 그리도록 만든다. 그 일관성이 가게의 품격이다.

4부

장바구니

31. 두 가지 장바구니

장바구니에는 두 가지 길이 있다. 세션에 둘 것인가, 데이터베이스에 둘 것인가.

세션은 가볍다. 손님이 주민등록증을 내밀지 않아도 책을 담을 수 있다. 다만 브라우저를 바꾸면 사라지고, 며칠 뒤 다시 와도 사라진다.

데이터베이스는 무겁지만 끈질기다. 같은 사람이 다른 기기에서 와도 자기 장바구니를 본다. 추천 시스템의 자료가 되기도 한다.

이 책은 두 가지를 함께 쓴다. 손님일 때는 세션, 로그인하는 순간 그 세션의 장바구니를 데이터베이스로 옮긴다. 두 길의 좋은 점만 가져오는 셈이다.

기억해 둘 단어: 손님 장바구니(*guest cart*)와 회원 장바구니(*user cart*). *머지(merge)*는 둘을 합치는 일이다.

다음 페이지에서 우리는 그 두 장바구니가 어떤 사물로 이루어져 있는지를 그린다.

32. Cart 도메인

도메인은 세 클래스다. Cart, CartItem, CartFactory.

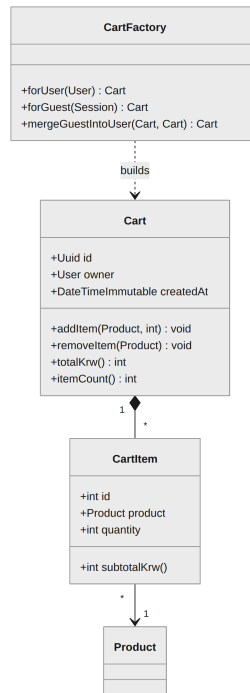


그림 5. 장바구니의 클래스 지도

Cart는 한 사람의 장바구니 한 개. CartItem은 그 장바구니 안에 든 한 줄(어떤 책 몇 권). CartFactory는 손님인지 회원인지를 보고 알맞은 Cart를 가져오는 작은 일꾼이다.

우리는 두 가지 결정에 마음을 쓴다. 첫째, 합계 계산은 어디서 하는가. 둘째, 장바구니가 비었다는 사실은 어떻게 다루는가.

합계는 Cart 안에서 한다. 컨트롤러도, 템플릿도 가격을 더하지 않는다. 가격을 더하는 일은 도메인의 책임이다. 그래야 같은 답을 어디서나 얻는다. 비었다는 사실은 다음 장에서 상태로 다룬다.

33. 장바구니의 상태

장바구니 한 개는 시간을 따라 이런 자리들을 옮겨 다닌다.

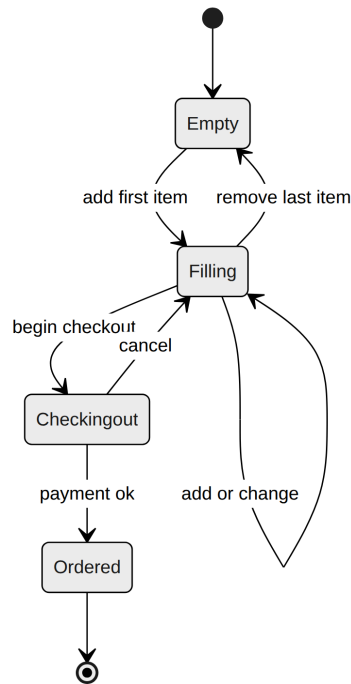


그림 6. 한 장바구니가 거치는 자리들

비어 있다가, 한 권을 담으면 채워진다. 결제로 들어가면 잠깐 잠금되고, 결제가 끝나면 주문이 된다. 결제를 취소하면 다시 채워진 상태로 돌아온다.

이 그림은 단순해 보이지만, 코드에서 두 가지 일을 막는다. 결제 중인 장바구니에 새 책을 담는 일과, 이미 주문이 된 장바구니를 다시 결제하는 일이다. 우리는 코드로 이 그림을 지킬 것이다.

심포니에는 Workflow 컴포넌트라는 더 정교한 도구가 있다. 6부에서 주문 상태를 다룰 때 부른다. 장바구니에는 단순한 status 컬럼으로 시작한다.

34. CartFactory — 손님과 회원

장바구니를 가져오는 길은 한 곳으로 모아 둔다.

```
// src/Cart/CartFactory.php
namespace App\Cart;

use App\Entity\Cart;
use App\Entity\User;
use App\Repository\CartRepository;
use Symfony\Component\HttpFoundation\RequestStack;

class CartFactory
{
    public function __construct(
        private CartRepository $carts,
        private RequestStack $requests,
    ) {}

    public function current(?User $user): Cart
    {
        if ($user) {
            return $this->carts->findActiveFor($user)
                ?? $this->carts->openFor($user);
        }
        $session = $this->requests->getSession();
        $id = $session->get('cart_id');
        return $id
            ? $this->carts->find($id) ?? $this->newGuest($session)
            : $this->newGuest($session);
    }
}
```

이 한 클래스가 “지금 이 사람의 장바구니”를 한 줄로 답한다. 컨트롤러는 더 이상 손님인지 회원인지를 묻지 않는다.

35. 답기

```
class AddToCartController extends AbstractController
{
    #[Route('/cart/add/{slug}',
        name: 'cart_add', methods: ['POST'])]
    public function __invoke(
        string $slug,
        Request $request,
        ProductRepository $products,
        CartFactory $factory,
        EntityManagerInterface $em,
    ): Response {
        $product = $products->findBySlug($slug);
        if (!$product) {
            throw $this->createNotFoundException();
        }
        $qty = max(1, (int) $request->request->get('qty', 1));

        $cart = $factory->current($this->getUser());
        $cart->addItem($product, $qty);

        $em->persist($cart);
        $em->flush();

        $this->addFlash('success', '장바구니에 담았습니다. ');
        return $this->redirectToRoute('cart_show');
    }
}
```

답는 일의 핵심은 한 줄, `$cart->addItem($product, $qty)` 이다. 같은 책을 두 번 담으면 수량만 늘린다. 그 결정은 Cart 안에서 한다. 컨트롤러는 모르는 일이다.

36. 빼기와 수량 변경

```
class UpdateCartController extends AbstractController
{
  #[Route('/cart/update', name: 'cart_update',
    methods: ['POST'])]
  public function __invoke(
    Request $request,
    CartFactory $factory,
    EntityManagerInterface $em,
  ): Response {
    $cart = $factory->current($this->getUser());
    $changes = $request->request->all('qty');

    foreach ($changes as $itemId => $qty) {
      $cart->setQuantity((int) $itemId, (int) $qty);
    }

    $em->flush();
    $this->addFlash('success', '장바구니를 갱신했습니다. ');
    return $this->redirectToRoute('cart_show');
  }
}
```

수량을 0으로 바꾸면 자동으로 빠진다. 그 결정도 Cart 안에서 한다. 컨트롤러는 품이 적은 그대로를 도메인에 넘기기만 한다.

이 패턴이 익숙해지면, 컨트롤러는 점점 짧아지고 도메인이 점점 두꺼워진다. 그 두꺼움이 우리가 이 가게에서 가진 “논리”다.

37. 합계 계산 — Money 객체

가격을 `int`로 다루는 일은 처음에는 편하다. 곧 미세한 오류가 보이기 시작한다. $0.1+0.2$ 가 0.3 이 아니라는 사실. 그래서 우리는 작은 Money 클래스를 둔다.

```
// src/Money/Krw.php
namespace App\Money;

final readonly class Krw
{
    public function __construct(public int $amount) {}

    public static function zero(): self { return new self(0); }

    public function add(Krw $other): self
    {
        return new self($this->amount + $other->amount);
    }

    public function multiply(int $n): self
    {
        return new self($this->amount * $n);
    }

    public function format(): string
    {
        return number_format($this->amount). '원';
    }
}
```

Cart의 `totalKrw()`는 이제 정수 대신 Krw를 반환할 수 있다. 화폐를 정수로 가지면서도 의미를 잃지 않는 방법이다.

한국 원화에서는 소수점이 거의 등장하지 않으므로 `int`로 충분하다. 다국어 통화로 가게가 자라면 그때 BCMath를 부른다.

38. 장바구니 머지

로그인하는 순간, 손님 장바구니가 회원 장바구니로 옮겨가야 한다. 이벤트로 부드럽게 처리한다.

```
// src/EventListener/MergeCartOnLogin.php
namespace App\EventListener;

use App\Cart\CartFactory;
use App\Entity\User;
use Symfony\Component\EventDispatcher\Attribute\AsEventListener;
use Symfony\Component\HttpFoundation\RequestStack;
use Symfony\Component\Security\Http\Event\InteractiveLoginEvent;

#[AsEventListener]
final class MergeCartOnLogin
{
    public function __construct(
        private CartFactory $factory,
        private RequestStack $requests,
    ) {}

    public function __invoke(InteractiveLoginEvent $event): void
    {
        $user = $event->getAuthenticationToken()->getUser();
        if (!$user instanceof User) return;

        $session = $this->requests->getSession();
        $guestId = $session->get('cart_id');
        if ($guestId) {
            $this->factory->mergeGuestIntoUser($guestId, $user);
            $session->remove('cart_id');
        }
    }
}
```

로그인 이벤트가 한 번 일어날 때마다, 이 작은 클래스가 깨어나서 머지를 한다. 컨트롤러는 모르는 일이다.

39. 미니 카트 위젯

헤더에 항상 작은 장바구니 표시가 떠 있어야 한다. 매 컨트롤러에서 직접 셸을 하면 일이 두 배가 된다. Twig 함수를 만든다.

```
// src/Twig/CartExtension.php
namespace App\Twig;

use App\Cart\CartFactory;
use Symfony\Bundle\SecurityBundle\Security;
use Twig\Extension\AbstractExtension;
use Twig\TwigFunction;

class CartExtension extends AbstractExtension
{
    public function __construct(
        private CartFactory $factory,
        private Security $security,
    ) {}

    public function getFunctions(): array
    {
        return [
            new TwigFunction('cart_count', $this->count(...)),
        ];
    }

    public function count(): int
    {
        $cart = $this->factory->current(
            $this->security->getUser(),
        );
        return $cart->itemCount();
    }
}
```

이제 어디서든 `{{ cart_count() }}`을 부르기만 하면 된다. 라라벨의 `@inject` + Blade와 비슷한 감각이다.

40. 빈 장바구니의 친절

장바구니가 빈 페이지를 본 손님은 다음 한 걸음을 알고 싶어 한다. 빈 화면이 단호한 거절처럼 보이지 않게 적자.

```
{# templates/cart/show.html.twig #}
{% extends 'base.html.twig' %}
{% block body %}
    <h1>장바구니</h1>
    {% if cart.itemCount == 0 %}
        <div class="empty">
            <p>아직 담은 책이 없어요.</p>
            <p>
                <a href="{{ path('catalog') }}">진열대로 가기</a>
            </p>
        </div>
    {% else %}
        <form method="post" action="{{ path('cart_update') }}">
            {% for item in cart.items %}
                {{ include('cart/_row.html.twig', {item: item}) }}
            {% endfor %}
            <p class="total">
                합계 {{ cart.totalKrw|number_format }}원
            </p>
            <button>주문으로</button>
        </form>
    {% endif %}
{% endblock %}
```

빈 장바구니는 막다른 길이 아니라, 다른 길로 향하는 표지판이어야 한다. 그 작은 친절이 가게의 인격이다.

41. 재고를 흘리지 않게

장바구니에 담긴 수량이 재고보다 많으면 어떻게 할까. 두 가지 결정이 있다. 담는 순간 막거나, 결제 직전에 막거나. 이 책방은 결제 직전에 막는다. 담는 일은 가벼워야 하고, 결제는 신중해야 하기 때문이다. 도메인 안에서.

```
class Cart
{
    /**
     * @return CartItem[] 재고가 부족한 항목들
     */
    public function itemsExceedingStock(): array
    {
        $bad = [];
        foreach ($this->items as $item) {
            if ($item->quantity > $item->product->getStock()) {
                $bad[] = $item;
            }
        }
        return $bad;
    }
}
```

결제 컨트롤러는 첫 줄에서 이 메서드를 부르고, 비어 있지 않으면 친절히 메시지와 함께 장바구니로 돌려보낸다. 도메인이 답하고, 컨트롤러는 그 답을 따른다.

5부

사 람 과 자 물 식

42. Symfony Security 풍경

심포니의 보안은 네 사물로 이루어져 있다.

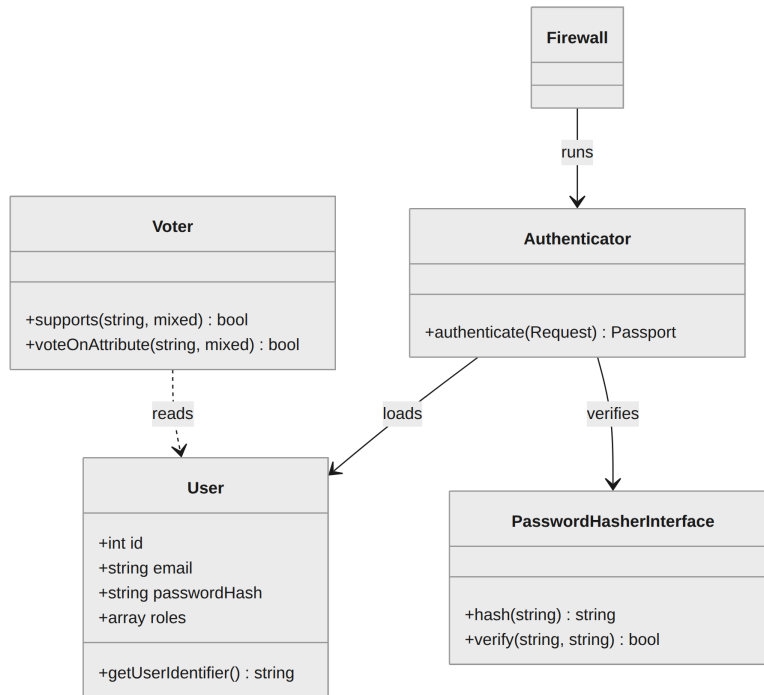


그림 7. 자물쇠를 만드는 네 사물

User는 누구인지를 담는 모델. **PasswordHasher**는 비밀번호를 단방향으로 굳히는 도구. **Authenticator**는 들어오는 요청을 보고 “이 사람은 누구다”를 결정한다. **Voter**는 “이 사람이 이 일을 해도 되는가”를 결정한다.

라라벨의 Auth와 Gate가 한 가족이라면, 심포니에서는 그 둘이 두 클래스로 나뉘어 자란다. 인증과 인가의 분리. 처음에는 번거롭지만, 큰 가게가 되면 이 분리가 코드를 가볍게 만든다.

Firewall은 그 모든 일이 일어나는 무대다. 어느 URL이 어느 인증을 쓸지를 적은 무대 설계도다.

43. User 엔티티

`make:user`로 골격을 만든다. 라라벨의 `php artisan make:user`와 같은 손이다.

```
php bin/console make:user
```

```
#[ORM\Entity]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 180, unique: true)]
    private string $email = '';

    #[ORM\Column]
    private array $roles = [];

    #[ORM\Column]
    private string $password = '';

    public function getRoles(): array
    {
        $roles = $this->roles;
        $roles[] = 'ROLE_USER';
        return array_unique($roles);
    }

    public function getUserIdentifier(): string
    {
        return $this->email;
    }
}
```

두 인터페이스가 약속을 정의한다. `UserInterface`는 “이 객체는 사용자다”, `PasswordAuthenticatedUserInterface`는 “이 사용자는 비밀번호로 들어온다”.

44. 비밀번호 해싱

비밀번호를 평문으로 저장하는 것은 가게 문을 열어 두는 일이다. 심포니에는 `PasswordHasher`가 있다.

```
use Symfony\Component>PasswordHasher\Hasher\UserPasswordHasherInterface;

class RegisterController extends AbstractController
{
    public function __invoke(
        Request $request,
        UserPasswordHasherInterface $hasher,
        EntityManagerInterface $em,
    ): Response {
        $user = new User();
        $form = $this->createForm(RegisterType::class, $user);
        $form->handleRequest($request);

        if ($form->isSubmitted() && $form->isValid()) {
            $plain = $form->get('plainPassword')->getData();
            $user->setPassword($hasher->hashPassword($user, $plain));
            $em->persist($user);
            $em->flush();
            return $this->redirectToRoute('login');
        }
        // ...
    }
}
```

해싱 알고리즘은 `config/packages/security.php`에서 한 줄로 정한다. 기본은 자동(`auto`)이며, PHP가 원하는 해시를 그때그때 고른다. 라라벨의 `Hash::make()`와 같은 일이다.

45. 회원가입 폼

```
// src/Form/RegisterType.php
namespace App\Form;

use App\Entity\User;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\Extension\Core\Type\EmailType;
use Symfony\Component\Form\Extension\Core\Type>PasswordType;
use Symfony\Component\Form\Extension\Core\Type\RepeatedType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use Symfony\Component\Validator\Constraints as Assert;

class RegisterType extends AbstractType
{
    public function buildForm(FormBuilderInterface $b, array $opts): void
    {
        $b->add('email', EmailType::class, ['label' => '이메일'])
            ->add('plainPassword', RepeatedType::class, [
                'type' => PasswordType::class,
                'mapped' => false,
                'first_options' => ['label' => '비밀번호'],
                'second_options' => ['label' => '비밀번호 확인'],
                'constraints' => [
                    new Assert\NotBlank(),
                    new Assert\Length(min: 8),
                ],
            ]);
    }

    public function configureOptions(OptionsResolver $r): void
    {
        $r->setDefaults(['data_class' => User::class]);
    }
}
```

`mapped: false`는 “이 필드는 엔티티에 저장하지 마라”는 뜻. 평문 비밀번호가 그대로 데이터베이스에 들어가지 않게 막는 작은 약속이다.

46. 로그인 폼

심포니 8에서는 폼 로그인을 한 명령어로 만든다.

```
php bin/console make:security:form-login
```

이 명령어가 `SecurityController`, `login.html.twig`, 그리고 `config/packages/security.php`의 firewall을 자동으로 갱신한다. 컨트롤러는 이렇게 생겼다.

```
class SecurityController extends AbstractController
{
    #[Route(path: '/login', name: 'login')]
    public function login(AuthenticationUtils $utils): Response
    {
        return $this->render('security/login.html.twig', [
            'last_username' => $utils->getLastUsername(),
            'error' => $utils->getLastAuthenticationError(),
        ]);
    }

    #[Route(path: '/logout', name: 'logout')]
    public function logout(): never
    {
        throw new \LogicException('intercepted by firewall');
    }
}
```

로그인 폼이 실제로 일을 처리하지 않는다는 점에 주목하자. 폼은 그저 input을 보여주고, 실제 인증은 firewall이 가로챈다. 로그아웃도 마찬가지다.

47. firewall.yaml 핵심

firewall은 어느 URL이 어떻게 인증되는지를 적는 무대 설계도다. 심포니 8은 PHP로 적는다.

```
// config/packages/security.php
use Symfony\Config\SecurityConfig;

return static function (SecurityConfig $sec) {
    $sec->passwordHasher(PasswordAuthenticatedUserInterface::class)
        ->algorithm('auto');

    $sec->provider('app_user_provider')
        ->entity()
        ->class(User::class)
        ->property('email');

    $main = $sec->firewall('main');
    $main->lazy(true)
        ->provider('app_user_provider')
        ->formLogin()
            ->loginPath('login')
            ->checkPath('login')
            ->defaultTargetPath('home');
    $main->logout()->path('logout');

    $sec->accessControl()
        ->path('^/admin')->roles(['ROLE_ADMIN']);
};
```

`lazy: true`는 “진짜로 사용자가 필요할 때만 세션을 깨워라”는 뜻. 익명 페이지의 속도가 빨라진다.

48. ROLE 관리

ROLE은 한 사용자에게 붙는 작은 깃발이다. `ROLE_USER`는 가입한 사람 모두, `ROLE_ADMIN`은 가게 주인. 우리 책방은 두 가지면 충분하다.

ROLE은 계층을 가질 수 있다. `config/packages/security.php`에서.

```
$sec->roleHierarchy('ROLE_ADMIN', ['ROLE_USER']);  
$sec->roleHierarchy('ROLE_OWNER', ['ROLE_ADMIN']);
```

이 한 줄이면 `ROLE_ADMIN`을 가진 사람은 자동으로 `ROLE_USER`이기도 하다. 권한이 늘어날수록 위로 올라가는 모양이다.

새 사용자가 가입할 때는 `ROLE_USER`가 자동으로 붙는다(우리 User의 `getRoles()`를 떠올리자). 첫 번째 관리자는 콘솔로 만든다.

```
php bin/console app:promote-admin admin@slow.book
```

그 명령어는 우리가 직접 만들 것이다. 7부에서 보자.

49. #[IsGranted]

컨트롤러 메서드 위에 한 줄 속성을 더하면 권한이 강제된다.

```
use Symfony\Component\Security\Http\Attribute\IsGranted;

class AdminProductController extends AbstractController
{
    #[Route('/admin/products/new', name: 'admin_product_new')]
    #[IsGranted('ROLE_ADMIN')]
    public function __invoke(/* ... */): Response
    {
        // ROLE_ADMIN이 아니면 이 코드는 절대 실행되지 않는다
    }
}
```

`#[IsGranted]`는 클래스 위에도, 메서드 위에도 둘 수 있다. 클래스 전체를 잠그고 싶다면 클래스 위에 둔다. 라라벨의 `middleware('auth')`와 같은 손이다.

인자도 함께 줄 수 있다. `#[IsGranted('ROLE_ADMIN', 'product')]`처럼 두 번째 인자를 주면 그 인자가 Voter에게 전달된다. Voter는 다음 페이지에서 만난다.

50. Voter — 도메인 권한

“이 사람이 이 주문을 볼 수 있는가” 같은 질문은 ROLE 하나로는 답할 수 없다. 그 주문이 그 사람의 것인지 알아야 하기 때문이다. 그 답을 적는 곳이 Voter다.

```
// src/Security/OrderVoter.php
namespace App\Security;

use App\Entity\Order;
use App\Entity\User;
use Symfony\Component\Security\Core\Authentication\Token\TokenInterface;
use Symfony\Component\Security\Core\Authorization\Voter\Voter;

class OrderVoter extends Voter
{
    public const VIEW = 'view';

    protected function supports(string $att, mixed $subject): bool
    {
        return $att === self::VIEW && $subject instanceof Order;
    }

    protected function voteOnAttribute(
        string $att, mixed $subject, TokenInterface $token,
    ): bool {
        $user = $token->getUser();
        if (!$user instanceof User) return false;
        return $subject->getCustomer()->getId() === $user->getId();
    }
}
```

컨트롤러에서는 한 줄로 묻는다.

```
$this->denyAccessUnlessGranted('view', $order);
```

51. 비밀번호 재설정

비밀번호 재설정의 흐름은 다음과 같다.

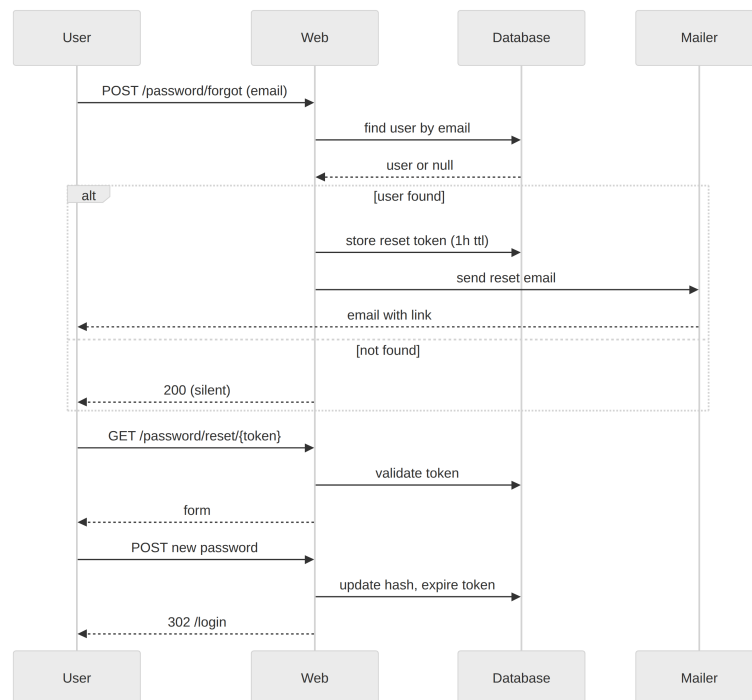


그림 8. 잊은 비밀번호를 다시 만나는 길

두 가지 결정에 마음을 쓰자.

첫째, 이메일이 데이터베이스에 없을 때도 “이메일로 보냈다”고 답한다. 그래야 공격자가 회원의 이메일을 추측해 알아낼 수 없다. 작은 거짓말이지만 친절한 거짓말이다.

둘째, 토큰의 수명은 1시간 정도가 좋다. 너무 길면 위험하고, 너무 짧으면 메일이 도착하기 전에 만료된다. 토큰은 한 번 쓰면 즉시 무효가 되어야 한다.

심포니는 `symfonycasts/reset-password-bundle`을 권한다. 위의 두 결정을 자동으로 처리한다. 우리가 직접 적어도 좋다. 그 코드는 단지 위의 시퀀스를 그대로 적는 일이다.

52. 로그아웃과 세션

로그아웃은 firewall이 잡는다. 우리는 라우트만 걸어 두면 된다.

```
$main->logout()  
    ->path('logout')  
    ->target('home')  
    ->invalidateSession(true);
```

`invalidateSession`은 로그아웃하면서 세션 전체를 비운다. 손님 장바구니가 남으면 곤란하다. 비워야 한다.

세션의 수명은 `config/packages/framework.php`에서 정한다.

```
$framework->session()  
    ->cookieLifetime(0)           // 브라우저 닫으면 사라짐  
    ->cookieSameSite('lax')  
    ->cookieSecure(true);        // https에서만
```

'기억하기' 기능을 더하고 싶다면 `rememberMe`를 firewall에 적는다. 이 책에서는 다루지 않는다. 짧은 책의 미덕은 빠진 부분에 있다.

6부

주 문

53. Order 도메인

주문은 장바구니의 다음 자리다. 시간이 멈춘 장바구니라고 생각해도 좋다.

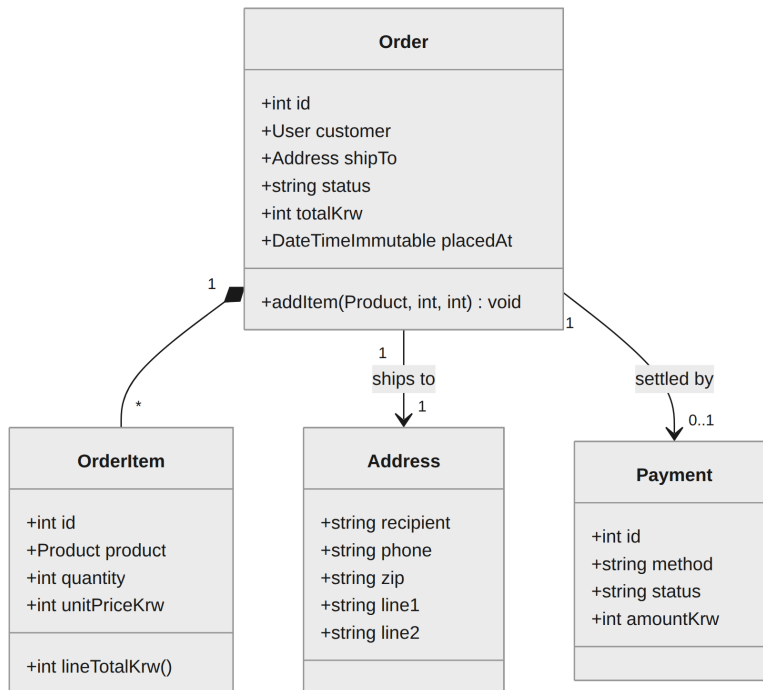


그림 9. 한 주문을 이루는 사물들

네 사물이 있다. Order는 주문 자체. OrderItem은 그 주문의 한 줄. Address는 배송지. Payment는 결제 기록. 이 네 가지가 함께 움직인다.

한 가지 약속에 마음을 쓰자. **OrderItem은 그 시점의 가격을 자기 안에 담는다.** 책의 가격이 나중에 바뀌어도, 그 주문의 줄에는 옛 가격이 남아 있다. 이 작은 결정이 회계의 순결함을 지킨다.

장바구니에서는 가격을 Product에서 매번 꺼냈다. 주문에서는 OrderItem이 자기 안에 가격을 가진다. 시간이 멈췄기 때문이다.

54. ER — 주문 데이터

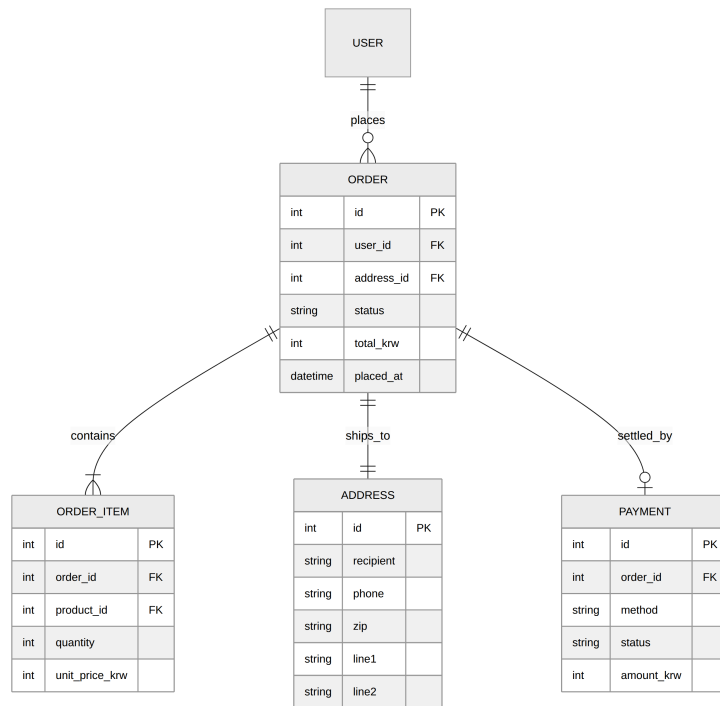


그림 10. 주문 데이터의 모양

네 테이블, 세 외래 키. 한 사용자가 여러 주문을 하고, 한 주문은 여러 줄을 가지고, 한 주문은 한 배송지로 향한다. 결제는 0개 혹은 1개. 결제를 하지 못한 채 취소된 주문도 우리는 데이터로 남긴다.

`unit_price_krw`가 `order_item`에 있는 점에 주목하자. 이 컬럼이 우리가 53쪽에서 했던 약속을 데이터베이스의 자리로 옮긴다. 회계가 열흘 뒤에도 같은 답을 낸다.

마이그레이션은 우리가 적은 엔티티에서 자동으로 만들어진다. 손으로 SQL을 적지 않는다. 다만 인덱스 한두 개는 직접 더하는 편이 좋다. `ix_order_user_id`, `ix_order_status`가 그것이다. 8부에서 본다.

55. 주문의 상태

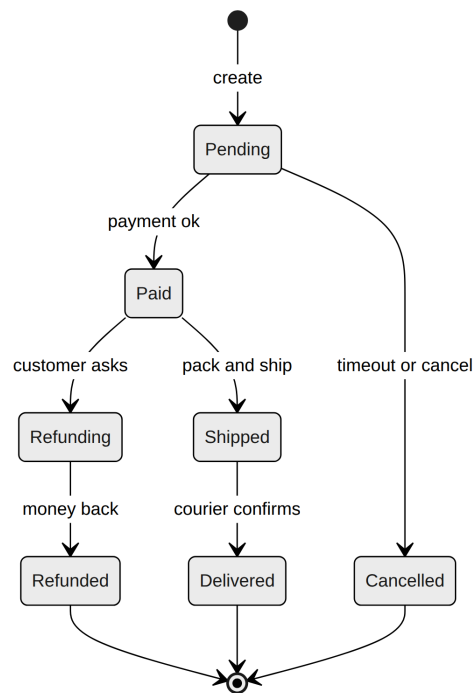


그림 11. 한 주문이 거치는 자리들

주문 한 건은 보통 다섯 자리를 거친다. `pending` (만들어졌지만 결제 전), `paid` (결제 완료), `shipped` (배송 시작), `delivered` (도착), `cancelled` (취소). 환불은 `refunding` 을 거쳐 `refunded` 로 간다.

이 그림에서 중요한 두 가지를 짚는다.

첫째, `pending` 에서 `shipped` 로 직접 갈 수 없다. 결제 없이 배송하지 않는다. 코드가 이 약속을 깨는 일을 막아야 한다.

둘째, `delivered` 에서 `cancelled` 로 갈 수 없다. 도착한 주문은 환불의 길로만 되돌릴 수 있다.

이 두 가지를 코드로 지킬 도구가 심포니의 `Workflow` 컴포넌트다. 61쪽에서 만난다.

56. 다단계 폼

주문은 보통 세 단계로 받는다. 주소, 결제 수단, 확인. 한 화면에 다 적으면 손님이 멀미를 한다.

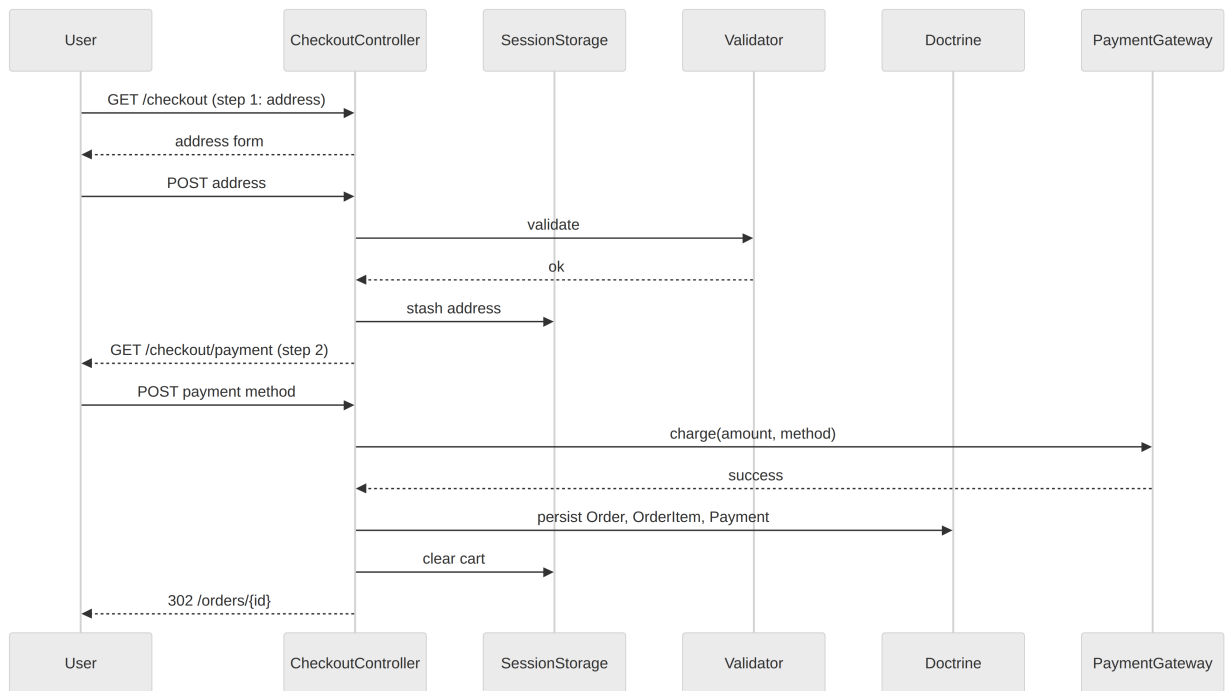


그림 12. 주문이 이루어지는 길

각 단계마다 폼이 나오고, 그 폼의 값이 세션 또는 임시 데이터베이스에 저장된다. 마지막 “확인” 버튼이 눌렸을 때 비로소 Order 객체가 만들어진다.

심포니 8에는 다단계 폼을 위한 `Form Wizard`라는 새 도구가 있다. 작은 책방인 우리는 손으로 짠다. 단계가 셋이고, 각 단계마다 컨트롤러 하나면 된다. 단계의 진행 상태는 세션에 둔다.

기억해 둘 단어: *idempotency*. 손님이 “확인”을 두 번 누르면 주문이 두 번 들어가지 않도록, 마지막 단계에는 토큰을 둔다. 트랜잭션과 함께 해결한다.

57. 배송 주소 폼

```
// src/Form/AddressType.php
class AddressType extends AbstractType
{
    public function buildForm(FormBuilderInterface $b, array $opts): void
    {
        $b->add('recipient', TextType::class, [
            'label' => '받는 분', 'constraints' => [new NotBlank()],
        ])
        $b->add('phone', TextType::class, [
            'label' => '연락처', 'constraints' => [
                new NotBlank(),
                new Regex('/^[0-9-]{9,15}$/'),
            ],
        ])
        $b->add('zip', TextType::class, [
            'label' => '우편번호',
            'constraints' => [new Regex('/^\d{5}$/')],
        ])
        $b->add('line1', TextType::class, ['label' => '주소'])
        $b->add('line2', TextType::class, [
            'label' => '상세 주소', 'required' => false,
        ]);
    }

    public function configureOptions(OptionsResolver $r): void
    {
        $r->setDefaults(['data_class' => Address::class]);
    }
}
```

제출이 끝나면 `$session->set('checkout_address', $form->getData())`로 잠시 보관한다. Address는 직렬화 가능한 단순한 객체여야 한다.

58. 결제 폼

실제 결제 게이트웨이 연동은 책의 영역을 넘는다. 여기서는 가짜 결제 게이트웨이로 흐름만 짚는다.

```
// src/Payment/FakeGateway.php
namespace App\Payment;

final class FakeGateway implements GatewayInterface
{
    public function charge(int $amountKrw, string $method): Charge
    {
        // 진짜라면 Stripe, Toss, IamPort 같은 곳을 부른다
        usleep(200_000); // 잠깐 기다리는 척
        return new Charge(
            id: bin2hex(random_bytes(8)),
            status: 'success',
            amountKrw: $amountKrw,
            method: $method,
        );
    }
}
```

인터페이스를 두는 이유는, 진짜 게이트웨이로 바꿀 때 이 클래스만 갈아 끼우면 되기 때문이다. 컨트롤러는 `GatewayInterface`만 받는다. 의존성 역전이라는 옛 약속이 여기서도 통한다.

현실의 결제는 비동기 콜백을 받는다. 그 콜백 처리는 9부의 운영 장에서 짧게 짚는다.

59. Cart에서 Order로

주문을 만드는 일은 한 메서드에 모은다.

```
// src/Order/PlaceOrder.php
namespace App\Order;

final readonly class PlaceOrder
{
    public function __construct(
        private EntityManagerInterface $em,
        private GatewayInterface $gateway,
    ) {}

    public function __invoke(
        Cart $cart, Address $address, string $method,
    ): Order {
        $order = new Order($cart->getOwner(), $address);
        foreach ($cart->getItems() as $item) {
            $order->addItem(
                $item->getProduct(),
                $item->getQuantity(),
                $item->getProduct()->getPriceKrw(),
            );
        }
        $charge = $this->gateway->charge($order->totalKrw(), $method);
        $order->markPaid($charge);

        $this->em->persist($order);
        $cart->markOrdered();
        $this->em->flush();
        return $order;
    }
}
```

이 클래스는 한 번 호출에 한 번의 일만 한다. 컨트롤러는 이 클래스를 받기만 하면 된다.

60. 트랜잭션

59쪽의 `PlaceOrder`는 미세하게 위험하다. 결제는 성공했는데 데이터베이스 저장이 실패하면, 손님은 돈을 냈는데 주문이 없다. 트랜잭션으로 묶자.

```
public function __invoke(Cart $cart, Address $a, string $m): Order
{
    return $this->em->wrapInTransaction(
        function () use ($cart, $a, $m) {
            $order = new Order($cart->getOwner(), $a);
            // ... 위와 동일 ...
            return $order;
        },
    );
}
```

`wrapInTransaction`은 콜백 안의 모든 일을 한 트랜잭션으로 묶는다. 예외가 나면 자동으로 롤백한다. 라라벨의 `DB::transaction()`과 같은 일이다.

다만 결제 호출은 트랜잭션 밖이 더 옳다. 결제는 외부 시스템에 영향을 주는데, 트랜잭션 안에 두면 우리 데이터베이스가 잠긴 동안 외부 호출을 기다리게 된다. 결제는 트랜잭션 직전에, 저장만 트랜잭션 안에. 작은 차이지만 자주 본다.

61. Workflow 컴포넌트

주문의 상태 전이는 우리가 그림으로 그렸다. 그 그림을 코드로 적는 도구가 Workflow다.

```
// config/packages/workflow.php
$framework->workflows()->workflows('order')
    ->type('state_machine')
    ->markingStore()->property('status')
    ->supports([Order::class])
    ->places(['pending', 'paid', 'shipped', 'delivered', 'cancelled'])
    ->transitions([
        'pay' => ['from' => 'pending', 'to' => 'paid'],
        'cancel' => ['from' => 'pending', 'to' => 'cancelled'],
        'ship' => ['from' => 'paid', 'to' => 'shipped'],
        'deliver' => ['from' => 'shipped', 'to' => 'delivered'],
    ]);
```

```
use Symfony\Component\Workflow\WorkflowInterface;

class ShipOrder
{
    public function __construct(
        private WorkflowInterface $orderWorkflow,
    ) {}

    public function __invoke(Order $order): void
    {
        $this->orderWorkflow->apply($order, 'ship');
    }
}
```

`apply`는 잘못된 전이를 자동으로 막는다. `delivered`인 주문에 `cancel`을 시도하면 예외가 난다.

62. 주문 메일 (Mailer)

주문이 들어오면 손님에게 확인 메일을 보낸다.

```
use Symfony\Bridge\Twig\Mime\TemplatedEmail;
use Symfony\Component\Mailer\MailerInterface;

class SendOrderConfirmation
{
    public function __construct(private MailerInterface $mailer) {}

    public function __invoke(Order $order): void
    {
        $email = (new TemplatedEmail())
            ->from('hello@slow.book')
            ->to($order->getCustomer()->getEmail())
            ->subject('주문이 접수되었습니다.')
            ->htmlTemplate('email/order_confirmation.html.twig')
            ->context(['order' => $order]);
        $this->mailer->send($email);
    }
}
```

개발 환경에서는 `MAILER_DSN=null://null`으로 두면 콘솔에 로그만 남고 실제로는 발송되지 않는다. 운영에서는 SMTP, SendGrid, Mailgun 등을 한 줄로 바꾼다. 라라벨의 `Mail::to()`와 거의 같은 손이다.

63. Messenger — 비동기

메일 발송을 컨트롤러 안에서 하면, 손님이 “확인” 버튼을 누른 뒤 1초쯤 멈춰 보인다. 메일은 큐로 흘리는 편이 옳다.

```
// src/Message/SendOrderConfirmation.php
namespace App\Message;

final readonly class SendOrderConfirmation
{
    public function __construct(public int $orderId) {}
}
```

```
// src/MessageHandler/SendOrderConfirmationHandler.php
use Symfony\Component\Messenger\Attribute\AsMessageHandler;

#[AsMessageHandler]
final class SendOrderConfirmationHandler
{
    public function __construct(
        private OrderRepository $orders,
        private MailerInterface $mailer,
    ) {}

    public function __invoke(SendOrderConfirmation $msg): void
    {
        $order = $this->orders->find($msg->orderId);
        // ... TemplatedEmail 만들어서 보내기 ...
    }
}
```

```
$bus->dispatch(new SendOrderConfirmation($order->getId()));
```

메시지를 던지면 즉시 반환되고, 백그라운드 워커가 그걸 처리한다. 라라벨의 `Job::dispatch()` 와 같은 일이다.

64. Scheduler — 자동화

매일 새벽에 “3일 이상 미배송 주문이 있는지” 살펴서 우리에게 메일을 보내자. Scheduler가 그 일을 한다.

```
use Symfony\Component\Scheduler\Attribute\AsSchedule;
use Symfony\Component\Scheduler\RecurringMessage;
use Symfony\Component\Scheduler\Schedule;
use Symfony\Component\Scheduler\ScheduleProviderInterface;

#[AsSchedule('default')]
final class DailySchedule implements ScheduleProviderInterface
{
    public function getSchedule(): Schedule
    {
        return (new Schedule())->add(
            RecurringMessage::cron(
                '0 6 * * *',
                new CheckUnshippedOrders(),
            ),
        );
    }
}
```

`CheckUnshippedOrders`는 우리가 만든 메시지다. 매일 아침 6시에 큐로 들어가고, 핸들러가 그것을 받아서 늦은 주문 목록을 만들어 우리에게 보낸다. 라라벨의 `Schedule::call()->daily()`와 같은 일이다.

워커는 `php bin/console messenger:consume scheduler_default`로 띄운다. 운영에서는 `systemd` 또는 `supervisor`로 살린다.

65. 주문 내역 보기

손님은 자기의 주문을 다시 보고 싶어 한다. 우리는 50쪽의 OrderVoter를 다시 부른다.

```
class ListMyOrdersController extends AbstractController
{
  #[Route('/orders', name: 'orders')]
  #[IsGranted('ROLE_USER')]
  public function __invoke(OrderRepository $orders): Response
  {
    return $this->render('order/index.html.twig', [
      'orders' => $orders->findBy(
        ['customer' => $this->getUser()],
        ['placedAt' => 'DESC'],
      ),
    ]);
  }
}

class ShowOrderController extends AbstractController
{
  #[Route('/orders/{id}', name: 'order_show')]
  public function __invoke(Order $order): Response
  {
    $this->denyAccessUnlessGranted('view', $order);
    return $this->render('order/show.html.twig', [
      'order' => $order,
    ]);
  }
}
```

두 컨트롤러가 깔끔히 권한을 분리한다. “내 주문 목록”은 ROLE_USER 전체가 볼 수 있고, “이 주문 하나”는 그 주문의 주인만 볼 수 있다.

7부

관 리 자

66. 관리자의 두 길

관리자 화면에는 두 가지 길이 있다. 직접 만들 것인가, EasyAdmin을 부를 것인가.

EasyAdmin은 짧은 시간에 잘 작동하는 어드민을 만들어 준다. 엔티티마다 자동으로 목록·생성·수정·삭제 페이지가 생긴다. 라라벨에서 Filament나 Backpack을 썼던 경험을 떠올리면 된다.

이 책은 직접 만드는 길을 간다. 이유는 두 가지다. 첫째, 일이 적은 가게에서 외부 패키지 한 개는 무겁다. 둘째, 직접 만들면 어드민이 어떻게 생겼는지를 알게 된다. EasyAdmin은 그 다음에 부르자.

다만 이런 결정은 가게마다 다르다. 직원이 늘면 EasyAdmin이 더 빠르게 답을 준다. 둘을 비교한 표 하나를 적어 두자.

—	직접 만들기	EasyAdmin
초기 시간	오래	짧게
커스터마이징	자유	제약
코드량	많음	적음
학습	심포니	EasyAdmin

67. 어드민 라우트와 firewall

어드민의 모든 URL은 `/admin`으로 시작하게 한다. firewall에서 한 줄로 잠근다.

```
$sec->accessControl()  
    ->path('^/admin')  
    ->roles(['ROLE_ADMIN']);
```

이제 `/admin/...`은 `ROLE_ADMIN` 없이는 접근할 수 없다. 한 줄이 한 우주를 잠근다.

컨트롤러는 단순한 invokable 클래스로 적는다. 클래스 위에 한 번만 권한을 적어 두면 메서드별로 적을 일이 없다.

```
#[Route('/admin/products', name: 'admin_product_index')]  
#[IsGranted('ROLE_ADMIN')]  
class AdminProductIndexController extends AbstractController  
{  
    public function __invoke(ProductRepository $products): Response  
    {  
        return $this->render('admin/product/index.html.twig', [  
            'products' => $products->findBy([],  
                ['createdAt' => 'DESC']),  
        ]);  
    }  
}
```

이 패턴이 모든 어드민 컨트롤러의 뼈대다. 다음 페이지부터는 그 뼈대 위에 살을 붙인다.

68. 상품 CRUD

```
#[Route('/admin/products/new', name: 'admin_product_new')]
#[IsGranted('ROLE_ADMIN')]
class AdminProductNewController extends AbstractController
{
    public function __invoke(
        Request $request, EntityManagerInterface $em,
    ): Response {
        $product = new Product();
        $form = $this->createForm(ProductType::class, $product);
        $form->handleRequest($request);
        if ($form->isSubmitted() && $form->isValid()) {
            $product->setSlug(slugify($product->getName()));
            $em->persist($product);
            $em->flush();
            $this->addFlash('success', '책을 추가했습니다.');
```

```
            return $this->redirectToRoute('admin_product_index');
        }
        return $this->render('admin/product/new.html.twig', [
            'form' => $form,
        ]);
    }
}
```

수정과 삭제는 같은 모양에서 인자만 바뀐다. 수정은 `Product $product`를 받고, 삭제는 두 줄. `$em->remove($product); $em->flush();` CRUD 네 글자가 네 컨트롤러로 옮겨 간 셈이다.

69. 이미지 업로드

책 표지를 업로드하는 일은 작은 일이지만 마음을 써야 한다. 보안과 저장소 문제가 함께 있기 때문이다.

```
$b->add('cover', FileType::class, [
    'mapped' => false,
    'required' => false,
    'constraints' => [
        new File([
            'maxSize' => '4M',
            'mimeType' => ['image/jpeg', 'image/png', 'image/webp'],
            'mimeTypeMessage' => '이미지 파일만 올릴 수 있습니다.',
        ]),
    ],
]);
```

```
$file = $form->get('cover')->getData();
if ($file) {
    $name = bin2hex(random_bytes(8)).'.'.$file->guessExtension();
    $file->move(
        $this->getParameter('kernel.project_dir').'/public/uploads',
        $name,
    );
    $product->addImage(new ProductImage($name, 0));
}
```

파일 이름은 절대 손님이 준 이름을 그대로 쓰지 않는다. 임의의 16글자로 새로 지어 준다. 디렉터리 트래버설을 막는 작은 약속이다.

가게가 자라면 VichUploaderBundle을 부르거나, S3로 옮긴다. 처음에는 `public/uploads`로 충분하다.

70. 재고 관리

재고는 한 컬럼이지만 다루는 데 마음을 써야 한다. 두 사람이 동시에 마지막 한 권을 사면 어떻게 될까.

```
// src/Stock/StockKeeper.php
namespace App\Stock;

use App\Entity\Product;
use Doctrine\ORM\EntityManagerInterface;

final readonly class StockKeeper
{
    public function __construct(private EntityManagerInterface $em) {}

    public function decrement(Product $p, int $qty): void
    {
        $this->em->wrapInTransaction(function () use ($p, $qty) {
            $this->em->lock($p, LockMode::PESSIMISTIC_WRITE);
            $this->em->refresh($p);
            if ($p->getStock() < $qty) {
                throw new InsufficientStock($p);
            }
            $p->setStock($p->getStock() - $qty);
        });
    }
}
```

`PESSIMISTIC_WRITE` 락은 “내가 다 쓸 때까지 다른 사람은 이 행을 건드릴 수 없다”는 약속이다. 동시에 들어온 두 주문 중 하나는 락을 기다린다. 마지막 한 권은 둘 중 한 사람만 산다.

재고는 그래서 가벼운 컬럼이 아니다. 책방의 정직성이 이 한 클래스에 모인다.

71. 주문 상태 변경

관리자는 주문을 “배송 중”으로 옮기고, 도착하면 “배송 완료”로 옮긴다. Workflow가 잘못된 전이를 막는다.

```
#[Route('/admin/orders/{id}/ship',
  name: 'admin_order_ship', methods: ['POST'])]
#[IsGranted('ROLE_ADMIN')]
class AdminShipOrderController extends AbstractController
{
  public function __invoke(
    Order $order,
    WorkflowInterface $orderWorkflow,
    EntityManagerInterface $em,
    MessageBusInterface $bus,
  ): Response {
    if (!$orderWorkflow->can($order, 'ship')) {
      $this->addFlash('error', '배송으로 보낼 수 없는 주문입니다.');
```

```
      return $this->redirectToRoute('admin_order_show',
        ['id' => $order->getId()]);
    }
    $orderWorkflow->apply($order, 'ship');
    $em->flush();
    $bus->dispatch(new SendShipNotification($order->getId()));
    return $this->redirectToRoute('admin_order_show',
      ['id' => $order->getId()]);
  }
}
```

상태가 바뀐 직후에 메시지를 던지는 모양에 주목하자. 배송 알림 메일이 큐에 쌓이고, 화면은 즉시 다음 페이지로 간다.

72. 권한 분리 — ROLE_ADMIN과 그 너머

가게가 자라면 “관리자”라는 한 깃발로는 부족해진다. 책을 등록하는 사람, 주문을 관리하는 사람, 정산을 보는 사람이 다른 사람이다. ROLE을 더 잘게 나누자.

```
$sec->roleHierarchy(  
    'ROLE_OWNER',  
    ['ROLE_ADMIN_ORDER', 'ROLE_ADMIN_PRODUCT', 'ROLE_ADMIN BILLING'],  
);
```

그러면 컨트롤러 위의 IsGranted도 더 정교해진다.

```
#[IsGranted('ROLE_ADMIN_PRODUCT')]  
class AdminProductNewController { /* ... */ }  
  
#[IsGranted('ROLE_ADMIN_ORDER')]  
class AdminShipOrderController { /* ... */ }
```

“오너”는 모든 권한을 자동으로 가진다. “상품 담당자”는 주문을 만질 수 없다. 권한이 잘게 나뉘어도 한 사람에게 여러 역할을 줄 수 있다는 점에서 유연하다.

처음부터 이렇게 나눌 필요는 없다. 한 사람이 다 보는 작은 책방에서는 ROLE_ADMIN 하나면 된다. 조직이 커질 때 위의 한 줄을 더하면 된다.

8부

화면

73. AssetMapper

라라벨에서 우리는 Vite를 만났다. 자바스크립트와 CSS를 한 데 묶어 빠르게 보내는 도구다. 심포니 8은 비슷한 자리에 `AssetMapper`를 둔다. 다른 점이 있다면, 빌드 단계가 없다는 것이다.

```
php bin/console importmap:require simple-stylesheets
```

이 명령어 한 번이면 `assets/` 폴더와 `importmap.php` 파일이 그 라이브러리를 알게 된다. 우리가 적는 자바스크립트는 그대로 브라우저에 가고, 브라우저는 ESM과 importmap을 이해한다.

```
// assets/app.js
import './styles/app.css';
import './controllers';
console.log('느린 책방 시작.');
```

```
{# templates/base.html.twig #}
{% block stylesheets %}
    {{ importmap('app') }}
{% endblock %}
```

번들링이 없으니 빌드가 빠르다. 빠른 만큼 변화를 즉시 본다. 단, 라이브러리가 ESM을 지원해야 한다. 다행히 2026년의 자바스크립트 생태계 대부분이 그렇다.

74. Stimulus 컨트롤러

Stimulus는 작은 자바스크립트 동작을 HTML에 묶는 도구다. “버튼을 누르면 ...” 같은 일이 손에 잡힌다.

```
// assets/controllers/quantity_controller.js
import { Controller } from '@hotwired/stimulus';

export default class extends Controller {
  static targets = ['input'];
  static values = { min: Number, max: Number };

  increment() {
    const next = +this.inputTarget.value + 1;
    if (next <= this.maxValue) this.inputTarget.value = next;
  }

  decrement() {
    const next = +this.inputTarget.value - 1;
    if (next >= this.minValue) this.inputTarget.value = next;
  }
}
```

```
<div data-controller="quantity"
      data-quantity-min-value="1"
      data-quantity-max-value="{{ p.stock }}">
  <button data-action="quantity#decrement">-</button>
  <input data-quantity-target="input" value="1">
  <button data-action="quantity#increment">+</button>
</div>
```

HTML이 동작을 알게 된다. JavaScript는 별도 파일에 그치고, 마크업은 그 동작을 가리킨다. 라라벨의 Alpine.js와 비슷한 자리에 있다.

75. Live Component — 검색 자동완성

심포니 UX의 LiveComponent는 라이브와이어와 같은 발상을 PHP로 가져온 것이다. 자바스크립트를 거의 적지 않고도 살아 있는 컴포넌트를 만든다.

```
composer require symfony/ux-live-component
```

```
// src/Twig/Components/SearchBar.php
namespace App\Twig\Components;

use App\Repository\ProductRepository;
use Symfony\UX\LiveComponent\Attribute\AsLiveComponent;
use Symfony\UX\LiveComponent\Attribute\LiveProp;
use Symfony\UX\LiveComponent\DefaultActionTrait;

#[AsLiveComponent]
final class SearchBar
{
    use DefaultActionTrait;

    #[LiveProp(writable: true)]
    public string $query = '';

    public function __construct(private ProductRepository $products) {}

    public function getResults(): array
    {
        return $this->query
            ? $this->products->searchByKeyword($this->query)
            : [];
    }
}
```

템플릿에서는 `<twig:SearchBar />`로 부른다. 사용자가 입력할 때마다 서버가 새 결과를 그려서 그 자리만 갱신한다. 자바스크립트는 거의 없다.

76. Turbo로 부드럽게

Turbo는 페이지 이동을 부드럽게 한다. 클릭하면 전체 새로고침 대신, 변경된 부분만 바뀐다. 사용자에게는 SPA처럼 느껴지지만, 우리는 평범한 컨트롤러를 적는다.

```
composer require symfony/ux-turbo
```

```
// assets/app.js
import * as Turbo from '@hotwired/turbo';
Turbo.start();
```

이 두 줄이면 거의 모든 클릭이 부드러워진다. 폼 제출도 마찬가지다. 새로고침 없이 결과가 바뀐다.

다만 두 가지에 주의한다. 첫째, 자바스크립트로 직접 DOM을 만진 부분은 페이지 이동 후에도 살아 있어야 한다.

`turbo:load` 이벤트를 듣자. 둘째, 외부 사이트로 가는 링크에는 `data-turbo="false"`를 둔다. Turbo가 가로채지 않게.

라라벨에서 Hotwire나 Inertia를 썼던 경험이 있다면, Turbo의 결은 익숙할 것이다. 다만 Turbo는 더 가볍다. 우리가 적는 코드는 평범한 트위그뿐이다.

77. 이미지 최적화

책방의 표지는 가게의 얼굴이다. 그러나 표지가 무거우면 가게가 느려진다. 이미지를 한 번만 변환해 두자.

```
composer require liip/imaginary-bundle
```

```
// config/packages/liip_imaginary.php
$imaginary->filterSets('thumb_360', [
    'jpeg_quality' => 85,
    'filters' => [
        'thumbnail' => ['size' => [360, 480], 'mode' => 'outbound'],
    ],
]);
$imaginary->filterSets('cover_900', [
    'jpeg_quality' => 85,
    'filters' => [
        'thumbnail' => ['size' => [900, 1200], 'mode' => 'outbound'],
    ],
]);
```

```
{# 템플릿에서 #}

```

처음 호출에서 변환된 파일이 만들어지고, 두 번째부터는 그 파일이 직접 서비스된다. 캐시는 `var/cache/imaginary`에 쌓인다.

WebP 포맷을 더하면 화질을 거의 잃지 않으면서 30%쯤 작아진다. `format: webp`를 한 줄 더하면 된다.

78. 다국어

가게가 자라서 영어를 함께 쓴다고 하자. 심포니의 Translation 컴포넌트가 도와준다.

```
// translations/messages.ko.yaml (또는 .php)
return [
    'cart.empty' => '아직 담은 책이 없어요.',
    'cart.go_browse' => '진열대로 가기',
];

// translations/messages.en.yaml
return [
    'cart.empty' => 'No books in your cart yet.',
    'cart.go_browse' => 'Browse the shelves',
];
```

```
{# 트위그에서 #}
<p>{{ 'cart.empty'|trans }}</p>
<a href="{{ path('catalog') }}">{{ 'cart.go_browse'|trans }}</a>
```

현재 로케일은 URL의 첫 부분(`/en/...`)이나 헤더로 결정한다. 라라벨의 Lang 파일과 거의 같은 모양이다.

다국어를 처음부터 도입할 필요는 없다. 가게가 한 나라에 머물 때는 하드코딩이 더 빠르다. 다만 그 가능성을 열어두는 일은 어렵지 않다. 키를 미리 정해 두는 작은 습관이 그 길이다.

9부

공공

79. 캐시

가게가 빨라지는 가장 짧은 길은 같은 답을 두 번 만들지 않는 일이다. 심포니의 Cache 컴포넌트가 그 일을 한다.

```
use Symfony\Contracts\Cache\CacheInterface;
use Symfony\Contracts\Cache\ItemInterface;

class FeaturedProducts
{
    public function __construct(
        private CacheInterface $cache,
        private ProductRepository $products,
    ) {}

    public function __invoke(): array
    {
        return $this->cache->get(
            'featured.products',
            function (ItemInterface $item) {
                $item->expiresAfter(600); // 10분
                return $this->products->findFeatured(8);
            },
        );
    }
}
```

캐시 어댑터는 `config/packages/cache.php`에서 정한다. 개발에서는 파일, 운영에서는 Redis가 보통이다.

HTTP 캐시도 잊지 말자. `$response->setSharedMaxAge(60)` 한 줄이면 정적인 페이지가 1분 동안 CDN에 머무른다. 라라벨의 `Cache::remember()`와 같은 손이다.

80. 로그

가게의 일을 기록하는 일은 가게의 정직성과 같다. 심포니는 Monolog를 쓴다. 라라벨과 같은 라이브러리다.

```
use Psr\Log\LoggerInterface;

class PlaceOrder
{
    public function __construct(
        private LoggerInterface $logger,
        // ...
    ) {}

    public function __invoke(Cart $cart, /* ... */): Order
    {
        $this->logger->info('order.placing', [
            'cart_id' => $cart->getId(),
            'user_id' => $cart->getOwner()?->getId(),
        ]);
        // ...
    }
}
```

채널을 나누면 로그가 정돈된다. `config/packages/monolog.php` 에서.

```
$monolog->channel('order');
$monolog->handler('order_file')
    ->type('rotating_file')
    ->path('%kernel.logs_dir%/order.log')
    ->maxFiles(14)
    ->channels()->elements(['order']);
```

운영에서는 stdout으로 보내고 그 위에서 외부 도구가 모은다. 컨테이너의 시대에는 그쪽이 더 일반적이다.

81. 테스트

테스트는 두 종류로 쪼갬. 단위와 기능. 단위는 도메인 클래스 하나의 약속을 검사한다. 기능은 컨트롤러를 통해 흐름 전체를 검사한다.

```
// tests/Cart/CartTest.php
class CartTest extends TestCase
{
    public function test_adding_same_product_increments_quantity(): void
    {
        $cart = new Cart(new User('a@b.c'));
        $book = new Product('느린 시', 12000);
        $cart->addItem($book, 1);
        $cart->addItem($book, 2);
        $this->assertSame(3, $cart->quantityOf($book));
        $this->assertSame(36000, $cart->totalKrw());
    }
}
```

```
// tests/Catalog/CatalogControllerTest.php
class CatalogControllerTest extends WebTestCase
{
    public function test_catalog_lists_products(): void
    {
        $client = static::createClient();
        $client->request('GET', '/books');
        $this->assertResponseIsSuccessful();
        $this->assertSelectorTextContains('h1', '진열대');
    }
}
```

실행은 `php bin/phpunit`. 라라벨의 `php artisan test`와 같은 손이다.

82. 쿼리 성능

가게가 자라면 쿼리가 느려지기 시작한다. 첫 의심은 항상 N+1이다.

예: 책 목록을 그릴 때 각 책의 첫 표지를 보여준다. 만약 표지가 lazy loading이라면, 책 12권에 13개의 쿼리가 나간다(목록 1 + 표지 12). 우리는 이것을 `JOIN FETCH`로 줄인다.

```
// ProductRepository
public function paginate(int $page, int $perPage): array
{
    return $this->createQueryBuilder('p')
        ->leftJoin('p.images', 'i')->addSelect('i')
        ->orderBy('p.createdAt', 'DESC')
        ->setFirstResult(($page - 1) * $perPage)
        ->setMaxResults($perPage)
        ->getQuery()->getResult();
}
```

`addSelect`가 핵심이다. 이미지를 함께 SELECT하지 않으면, JOIN만 걸렸을 뿐 N+1은 그대로 남는다.

심포니의 프로파일러는 한 페이지마다 몇 개의 쿼리가 나갔는지를 보여준다. 개발 환경에서 자주 들여다보자. 빨간 숫자가 보이면 거기가 우리의 다음 일이다.

인덱스는 자주 검색하는 컬럼에 둔다. `slug`, `email`처럼 unique한 컬럼은 자동으로 인덱스를 가진다. 그 외는 우리가 적는다.

83. 보안 검토

가게의 문을 잠그기 전에, 다섯 가지를 점검한다.

CSRF

심포니의 폼은 자동으로 CSRF 토큰을 둔다. 폼 밖에서 POST를 받는 길(예: 자바스크립트로 만든 폼)에는 직접 토큰을 검사한다. `$this->isCsrfTokenValid('action', $token)`.

XSS

Twig은 기본으로 모든 출력을 이스케이프한다. `{{ ... }}`는 안전하다. `{{ ...|raw }}`는 위험하다.

SQL 인젝션

QueryBuilder의 자리표(`:q`)를 항상 쓴다. 변수를 SQL 문자열에 직접 끼워 넣지 않는다.

비밀

비밀은 `.env.local` 또는 환경변수. `git log`를 한 번 검색해 비밀이 새어 나가지 않았는지 확인한다.

의존성

`symfony security:check` 또는 `composer audit`가 알려진 취약점을 알려준다. CI에서 매주 돌리자.

84. 배포

배포의 길은 여러 가지다. 짧은 가게에는 두 가지를 권한다.

| Symfony Cloud

`symfony cloud:deploy` 한 줄이면 끝. 빠르게 띄울 수 있고, 워커와 cron까지 한 번에 챙겨 준다. 작은 가게에 알맞다.

| Docker

Dockerfile 한 장을 짜고, Caddy나 Nginx 뒤에 PHP-FPM을 둔다.

```
# Dockerfile (요약)
FROM php:8.5-fpm-alpine
RUN install-php-extensions intl pdo_pgsql opcache
COPY composer.json composer.lock ./
RUN composer install --no-dev --optimize-autoloader
COPY . .
RUN php bin/console cache:warmup --env=prod
CMD ["php-fpm"]
```

Messenger 워커는 별개의 컨테이너로 띄운다. `php bin/console messenger:consume async --time-limit=3600`를 supervisor 또는 systemd로 살린다.

둘 다 어렵다면 첫 번째를 선택하자. 가게의 일은 책을 파는 일이지, 서버를 다스리는 일이 아니다.

85. 모니터링

운영의 마지막 한 발은 모니터링이다. 가게가 도는 동안 무엇이 잘못되고 있는지를 보는 일이다.

세 가지를 본다.

에러. Sentry 같은 서비스가 표준이다. 심포니는 한 줄이면 연결된다.

```
composer require sentry/sentry-symfony
```

응답 시간. New Relic, Datadog, 또는 단순히 액세스 로그를 본다. 95퍼센트 응답 시간이 1초를 넘으면 어딘가가 느린 것이다.

큐 깊이. Messenger의 큐가 쌓이고 있다면 워커가 부족하다. `php bin/console messenger:stats`가 알려준다.

한 가지에 마음을 더하자. **알림 피로.** 모든 에러에 페이지를 받으면 우리는 곧 알림을 무시하게 된다. 정말 자다 일어날 만한 일에만 페이지를 받고, 나머지는 매일 아침 한 번 모아서 보자.

가장 좋은 모니터링은, 우리가 매일 한 번 가게를 사용해 보는 일이다.

가져갈 직관 · 두고 갈 습관 · 더 깊은 방들

가져갈 것

- 컨트롤러는 얇아야 한다는 감각.
- 라우트 모델 바인딩의 편안함.
- `.env`에 비밀을 두는 일.
- 마이그레이션을 코드와 함께 커밋하는 습관.
- 큐로 느린 일을 떼어 놓는 습관.

두고 갈 것

- 정적 메서드로 모든 일을 끝내려는 버릇.
- `make:` 명령어를 외워서 쓰는 습관.
- 헬퍼가 자동으로 글로벌에 떠 있을 것이라는 기대.
- 모델이 모든 도메인 일을 맡는 활성 레코드의 직관.

더 깊은 방들

Symfony Notifier(슬랙·SMS·푸시), API Platform(REST/GraphQL), Doctrine Migrations Diff와 customizing, EasyAdmin, Symfony UX의 컴포넌트(Map, Chartjs, Cropperjs), Profiler 분석, Blackfire 프로파일링, OpenSwoole 같은 비동기 런타임. 가게가 자라면 들어갈 방들이다.

각 방에 들어갈 때마다 한 권의 책이 더 필요할 수 있다. 그러나 그 모든 방의 골격은 우리가 이미 본 것이다. 받고, 옮기고, 꺼내고, 실행하고, 알린다.

닫는 글 · 콜로폰

백 쪽이 짧다고 느꼈다면 그건 잘된 일이다. 책은 짧을수록 다시 펴게 되고, 다시 펴는 책일수록 친구가 된다.

심포니는 우리에게 묻는 도시다. 처음에는 묻는 일이 피곤하다. 점점 자기만의 답이 생기고, 그 답들이 모여서 우리가 만들고 싶은 모양이 된다. 라라벨이 길러 준 직관은, 그 답을 짧게 만든다.

이 책으로 만든 느린 책방이 누군가의 가게를 시작하는 데 한 줄이라도 도움이 되었다면, 그것으로 충분하다. 다음 책은 더 짧을 수도 있다.

— jaywoo, 2026년 5월

콜로폰

본문	Noto Sans KR
코드	Roboto Mono
조판	htmlasitis 0.1.0
다이어그램	mermaid.js (PNG로 사전 렌더)
인쇄	A4 · 100면
저자	jaywoo · 2026